AttaCode: Attribution-Guided Adversarial Code Generation via Jailbreak-Enabled Prompts against Vulnerability Detection Models

Tongyang Xu, Jingyuan Li, Jiawei Xiong, Ju Jia, Yueming Wu, Siqi Ma, *Member, IEEE*, Guang Cheng, *Member, IEEE*, and Robert H. Deng, *Fellow, IEEE*

***Abstract*—Learning-based vulnerability detectors, particularly those built on pre-trained code models, are increasingly used in software assurance, yet they remain susceptible to semantics-preserving adversarial transformations. Although such attacks help expose and mitigate model weaknesses, existing methods use narrow transformation spaces, lack fine-grained guidance, or fragment code to circumvent large language model (LLM) safeguards, increasing detector queries and risking broken vulnerability-relevant dependencies. We propose AttaCode, an attribution-guided adversarial code generation scheme that prioritizes transformations using information from the detector and performs jailbreak-enabled LLM generation over complete source code. AttaCode aggregates gradient-based token attributions onto abstract syntax tree nodes to rank 25 semantics-preserving transformations. It then optimizes placeholder-based jailbreak prompts through iterative mutation to reduce safety refusals during subsequent generation, before instantiating them with the full program and dynamically selected transformations for iterative adversarial generation. Extensive experiments show that AttaCode outperforms state-of-the-art (SOTA) baselines, with an average attack success rate (ASR) above 80% while requiring 75%–96% fewer detector queries. It also achieves the highest ASR on average at every evaluated detector query budget. Its transformations preserve tested behavior for 89.9% of programs after three sequential edits, receive favorable naturalness ratings, and yield the lowest automated detection rates. Moreover, adversarial fine-tuning improves detector accuracy by 9.56–12.52 percentage points.**

***Index Terms*—Adversarial Code Generation, Attribution-Guided Perturbation, Jailbreak-Enabled Prompts, Semantic-Preserving Transformations, Vulnerability Detection.**

Received XXXX; revised XXXX; accepted XXXX. This work was supported in part by the National Natural Science Foundation of China under Grant 62402106 and Grant 62676088, in part by the Natural Science Foundation of Jiangsu Province of China under Grant BK20241272, in part by the "Zhishan" Scholars Programs of Southeast University under Grant 2242026RCB0031, and in part by the Start-Up Research Fund of Southeast University under Grant RF1028623129. *(Jingyuan Li and Jiawei Xiong contributed equally to this work.) (Corresponding author: Ju Jia.)*

T. Xu, J. Li, J. Xiong, J. Jia, and G. Cheng are with the School of Cyber Science and Engineering, Southeast University, Nanjing 211189, China. E-mail: tongyangxu@seu.edu.cn, lijingyuan@seu.edu.cn, xiongjiawei@seu.edu.cn, jiaju@seu.edu.cn, chengguang@seu.edu.cn.

Y. Wu is with the National Engineering Research Center for Big Data Technology and System, Services Computing Technology and System Lab, Hubei Engineering Research Center on Big Data Security, School of Cyber Science and Engineering, Huazhong University of Science and Technology, Wuhan 430074, China. E-mail: yuemingwu@hust.edu.cn.

S. Ma is with the School of Computing and Information Technology, University of Wollongong, Wollongong, NSW 2522, Australia. E-mail: siqim@uow.edu.au.

R. H. Deng is with the School of Information Systems, Singapore Management University, Singapore 188065. E-mail: robertdeng@smu.edu.sg.

I. Introduction

SOFTWARE vulnerability detection is essential for the security and reliability of modern systems, as vulnerabilities can lead to severe consequences such as data breaches and system compromise [1], [2]. With the growing scale and complexity of real-world software, automated vulnerability detection increasingly relies on learning-based models (e.g., UniXcoder [3]). Unlike traditional static analysis tools based on rigid heuristics, these code pre-trained models achieve superior performance by learning rich syntactic and semantic representations from massive code corpora [4]. Adversarial robustness remains a significant concern across learning-based systems [5], including code models [6]. Such attacks subtly perturb source code in ways that are benign or even imperceptible to human developers, yet can mislead models into misclassifying vulnerable code as secure. This susceptibility underscores the critical need for adversarial code generation, both to expose weaknesses in pre-trained models and provide high-quality samples for adversarial training, ultimately enhancing model dependability.

Existing adversarial code attacks can be grouped into four categories by their perturbation strategies. The first involves identifier substitution, which perturbs code by renaming program entities (e.g., variable or function names) with semantics-preserving alternatives [7], [8]. The second focuses on structural modifications by altering control flows or loop structures while preserving program logic [9], [10]. The third is dead code insertion, which introduces extraneous code segments without affecting program outputs [11], [12]. More recently, rapid development of LLMs has introduced a fourth category of adversarial attacks. Unlike the previous three categories relying on predefined heuristics, this new paradigm leverages the exceptional capabilities of LLMs in code reasoning and structured code generation [13] to craft adversarial code samples [14]. While these methods have advanced the field, most existing approaches prioritize attack success rates (ASR) over fine-grained analysis and preservation of source code semantic integrity, often resulting in inefficient query exploration or semantically incoherent examples.

Despite the progress of existing adversarial attacks, they still face significant challenges in practice. As illustrated in the top half of Figure 1, most existing attacks rely on a limited transformation space, estimate importance using

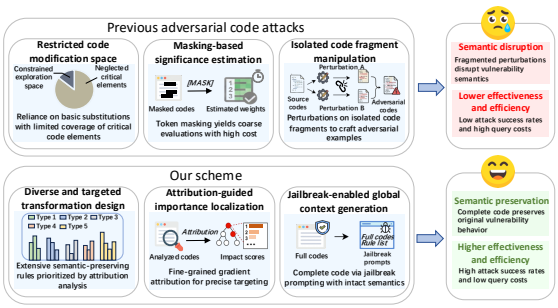


Fig. 1. Comparison between existing adversarial code attacks and the proposed AttaCode scheme.

masking-based strategies, and apply perturbations to isolated code fragments. First, existing methods employ a narrow set of semantics-preserving transformations (e.g., identifier renaming or dead code insertion), leaving many critical code elements underexplored, such as operators, keywords, and control-flow structures. Recent studies [15] indicate that these overlooked elements can significantly impact model predictions by altering code representations. Consequently, this restriction severely constrains the diversity of generated adversarial examples. Second, many attacks treat all code tokens with equal importance during perturbation, ignoring that certain tokens (e.g., sensitive API calls or pointer dereferences) are more influential to model predictions than others (e.g., variable names or comments). While token masking can estimate importance, it is computationally expensive and often ignores contextual interactions among tokens [16]. Moreover, statement-level importance analysis [17] is too coarse to capture fine-grained variations within a statement. This lack of fine-grained guidance results in inefficient searches that modify irrelevant code regions, increasing query overhead. Third, existing adversarial attacks commonly construct adversarial examples by applying perturbations to isolated code fragments, rather than operating on the full program context. Such fragment-level modifications often disrupt critical semantic dependencies that characterize vulnerability patterns, such as the dependency between a memory allocation and its subsequent deallocation. This issue becomes even more pronounced in recent LLM-based adversarial attacks. Due to strict safety alignment mechanisms in modern LLMs, prompts that explicitly request adversarial examples are often refused as potentially harmful [18], [19]. To bypass these restrictions, prior work fragments the source code into "target" and "context" sections and prompts the LLM to generate insertion snippets using preceding and succeeding code [14]. However, such partitioning further disrupts the semantic and structural integrity needed to represent vulnerabilities. Therefore, preserving global program context while bypassing safety restrictions is essential for effective LLM-based adversarial attacks.

To address these limitations, we propose **AttaCode**, an **Att**ribution-guided **a**dversarial **Code** generation scheme that couples detector-informed rule prioritization with jailbreak-enabled LLM generation under full program context. It maps fine-grained attribution signals through AST nodes to priorities over semantics-preserving transformations that guide iterative adversarial generation. As shown in the bottom half of Figure 1, AttaCode is designed to overcome these challenges. First, we perform fine-grained importance analysis via gradient attribution to identify the most influential tokens, avoiding repeated model probing required by masking-based methods. This guides the prioritization of our 25 semantics-preserving transformation rules (Section III-B) for targeted perturbation. Subsequently, to bypass safety alignment while retaining unfragmented code context, we design an automated jailbreak prompt construction system that iteratively mutates and evaluates prompts using refusal feedback. The resulting jailbreak prompts enable subsequent generation to operate on the full, unfragmented code context rather than partitioned snippets (Section III-C). Finally, AttaCode integrates these components in a prompt-driven iterative generation process. By constructing augmented prompts that populate jailbreak prompts with the full source code and a dynamically selected subset of transformation rules, our scheme efficiently guides LLMs to produce diverse adversarial examples (Section III-D). The main contributions of this paper are summarized as follows:

- We propose a fine-grained, attribution-guided importance analysis for targeted perturbations. Its default gradient-based estimator aggregates token-level gradients into abstract syntax tree (AST) nodes to identify critical code structures, thereby narrowing the search space and prioritizing effective transformation rules. Depending on the access conditions, alternative estimators can be used without changing the downstream pipeline.
- We develop an automated jailbreak prompt construction system to bypass the safety alignment of modern LLMs. Through iterative mutation guided by refusal feedback, we generate effective jailbreak prompts. In this way, AttaCode ensures that high-quality adversarial examples can be generated even from strictly aligned models without triggering refusal mechanisms.
- We integrate these components into a prompt-driven adversarial generation process that operates on the full source code. Unlike existing LLM-based attacks that rely on partitioned code snippets and may disrupt critical semantic dependencies, our scheme leverages jailbreak prompts to retain the unfragmented code context, thereby providing the LLM with the original program structure and cross-statement dependencies during generation.
- To the best of our knowledge, this is the first work to integrate attribution-guided rule prioritization with jailbreak-enabled LLM generation under full program context for adversarial code against vulnerability detectors. Our main technical contribution is coordinating AST-level rule prioritization, placeholder-based prompt optimization, and dynamically guided full-context generation. We conduct extensive experiments on multiple benchmark datasets against representative vulnerability detection models. Experiments show that AttaCode outperforms SOTA baselines, achieves an average ASR of over 80% while requiring up to 96% fewer queries to the detector, and improves model robustness by around 10%

via adversarial fine-tuning. Moreover, a controlled comparison shows that gradient guidance yields the highest attack effectiveness, strongest performance under small iteration budgets, and substantially lower attribution and query costs than importance estimation through masking.

Paper organization. The rest of this paper is organized as follows. Section II introduces the background and motivating example. Section III presents AttaCode. Section IV describes the experimental setup. Section V reports and analyzes the results. Section VI provides further discussion, Section VII reviews related work, and Section VIII concludes this paper.

II. Background and Motivation

A. Adversarial Attacks on Code Models

Code pre-trained models have achieved remarkable success in code intelligence tasks [20]–[22], such as vulnerability detection, code classification, and code summarization. However, recent studies have shown that these code models are vulnerable to adversarial attacks [23]. In the context of source code, such attacks involve small, syntactically valid modifications to the input code that preserve program functionality while misleading the model into making incorrect predictions. Unlike adversarial attacks in computer vision, code adversarial attacks must strictly satisfy programming language syntax and execution semantics. This strict adherence fundamentally constrains the design of effective perturbation strategies, which makes the generation of valid adversarial examples significantly more challenging.

Formally, let $f : \mathcal{X} \to \mathcal{Y}$ be a code model that maps a code snippet $x \in \mathcal{X}$ to a label $y = f(x) \in \mathcal{Y}$. Given an input code x with the ground-truth label y, an adversarial example x^{adv} is defined as a perturbed version of x that satisfies:

$$f(x^{adv}) \neq f(x) = y \tag{1}$$

The attack seeks a minimally perturbed x^{adv} that satisfies four requirements: (i) *attack effectiveness*, causing the target model to misclassify it, $f(x^{adv}) \neq y$; (ii) *syntactic validity*, ensuring that x^{adv} conforms to the programming language syntax and can be parsed or compiled; (iii) *semantic equivalence*, preserving the input-output behavior of x for the same inputs; and (iv) *perturbation stealthiness*, keeping the modification minimal and inconspicuous to human developers. These constraints jointly define the requirements for constructing valid adversarial examples and form the foundation of adversarial attack methods on code models. Adhering to these principles, our proposed attack scheme aims to improve attack effectiveness and query efficiency by focusing perturbations on the most influential code elements while preserving syntactic validity and program behavior.

B. Code Semantic-Preserving Transformation

Semantic-preserving transformations are widely applied in program analysis, optimization, and refactoring. In the context of adversarial code generation, such transformations provide a principled way to modify program structure while strictly preserving functional behavior. Unlike natural language, source code admits a range of structural and syntactic modifications that preserve program behavior, provided that the execution semantics remain unchanged. These transformations can be formally characterized as follows. Given two programs P and P' written in a programming language L, with original input space $\mathcal{I}$ and observable output space $\mathcal{O}$, we define P' as a semantic-preserving transformation of P when the following condition is satisfied:

$$\begin{aligned} f_P : \mathcal{I} \to \mathcal{O}, \qquad & f_{P'} : \mathcal{I} \times \Theta \to \mathcal{O}, \\ f_P(x) = f_{P'}(x, \theta), \quad & \forall x \in \mathcal{I},\ \forall \theta \in \Theta. \end{aligned} \tag{2}$$

where f_P and $f_{P'}$ denote the functions implemented by programs P and P', respectively, and Θ represents a set of auxiliary parameters introduced in P' that do not affect the functional behavior of the program.

Semantic-preserving transformations play a central role in adversarial code generation, offering an effective means to produce inputs functionally equivalent to the original yet capable of misleading model predictions. Although such transformations do not alter program behavior, they can substantially modify code representations learned by pre-trained models, thereby causing misclassification [24]–[26]. In this study, we further expand the transformation space by considering a broader, more diverse set of semantic-preserving transformations, enabling more flexible and powerful adversarial attacks.

C. Motivation Example

To illustrate the key insight of our scheme, we present a motivating example using a real-world vulnerable code snippet. As shown in Figure 2, the buffer pointer `req_buf` is deallocated by `g_free` and subsequently referenced in `fprintf`. This constitutes a classic use-after-free (UAF) vulnerability, which can lead to severe memory corruption.

EaTVul [14] generates adversarial code through a feature-driven insertion pipeline. It identifies important code features, categorizes them by syntactic type, and constructs prompts from partial preceding and succeeding contexts to ask an LLM for local insertion snippets. As shown in Figure 2(a), EaTVul identifies the deallocation operation `g_free(req_buf)`, the later output function `fprintf`, and the buffer pointer `req_buf`. However, the generated snippet operates on a newly allocated `dummy` buffer and is inserted after the deallocation of `req_buf`. Because this local insertion does not modify the dependency between `g_free(req_buf)` and the subsequent use of `req_buf`, the use-after-free pattern remains visible to the detector, and the modified function fails to mislead the model.

To address this limitation, AttaCode provides the generator LLM with an augmented jailbreak prompt containing the complete source code and attribution-prioritized transformation rules. For the example in Figure 2(b), the prompt directs the LLM to apply `WrapWithConstantCondition` to the vulnerability-relevant `fprintf` statement. As shown in Figure 2(b), the generated code wraps the sensitive `fprintf` statement within an always-true conditional block using `if ((cmd_id * 0) + 1)`. Importantly, this transformation is semantics-preserving because the condition always evaluates

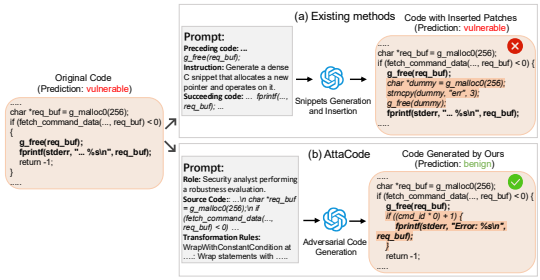


Fig. 2. Comparison of AttaCode and existing adversarial code generation methods on the same source code. (a) Existing methods use fragmented code contexts to construct prompts and generate inserted patches, failing to capture long-range semantic dependencies. (b) In contrast, AttaCode preserves full context through jailbreak-enabled prompting and successfully generates adversarial examples.

to true and the `fprintf` statement is executed under the same program behavior. Nevertheless, the additional control-flow structure alters the syntactic structure and token context around vulnerability-relevant operations, which can change the representations learned by the model. Consequently, the vulnerability detection model predicts the transformed code as non-vulnerable, demonstrating that full-context prompting can yield effective adversarial examples.

III. Proposed AttaCode Scheme

A. System Overview

Figure 3 illustrates the overall workflow of AttaCode, which consists of three main phases: fine-grained importance analysis, jailbreak prompt construction, and prompt-driven adversarial code generation.

- *Fine-Grained Importance Analysis:* This phase identifies the most influential source-code tokens for a vulnerability detection model. AttaCode primarily uses gradient-based attribution in this phase, and the resulting token importance scores guide the prioritization of semantics-preserving transformation rules in later stages.
- *Jailbreak Prompt Construction:* This phase aims to construct jailbreak prompts that mitigate safety refusals in aligned LLMs via an interactive mutation process. Its input is initial prompts, and the output is effective jailbreak prompts for the subsequent generation phase.
- *Prompt-Driven Adversarial Code Generation:* This phase performs prompt-driven adversarial code generation by leveraging the results of the previous phases. Specifically, transformation rules are ranked according to token importance and combined with the full source code and jailbreak prompts to form augmented prompts, which are then provided to the generator LLM.

AttaCode is distinguished by a unified attack design that translates attribution signals from the detector into structured guidance for jailbreak-enabled LLM generation. Specifically, it aligns AST-level importance scores with applicable transformations and uses the optimized jailbreak prompt to drive generation under full program context, steering each iteration toward influential code regions.

B. Fine-Grained Importance Analysis with Gradient Guidance

In adversarial code generation, tokens contribute unequally to model predictions. To target influential code regions, AttaCode adopts gradient-based attribution as a fine-grained estimator of token importance. When gradient access to the evaluated vulnerability detector is available, the estimator computes the gradient of the model output with respect to input token embeddings in a single backward pass. This yields a fine-grained importance score for each token to guide the prioritization of transformation rules in subsequent phases. The use of gradients is confined to computing the initial importance scores, and all subsequent stages rely solely on these scores without accessing gradients or internal model states.

Formally, given an input source code $x = [x_1, x_2, \ldots, x_n]$ of length n, it is first passed through a pre-trained vulnerability detection model $F(\cdot)$ to obtain the classification logits. Let $z = F(x) \in \mathbb{R}^2$ denote the logits, where z_{vul} and z_{sec} correspond to the "vulnerable" and "secure" labels, respectively. To mitigate gradient saturation caused by the Softmax normalization, we formulate the attribution target function $\mathcal{T}(x)$ as the difference between these logits:

$$\mathcal{T}(x) = z_{\text{vul}} - z_{\text{sec}} \tag{3}$$

After defining the attribution target $\mathcal{T}(x)$, the input code embeddings are $\mathbf{E} = [e_1, e_2, \ldots, e_n]$, where each $e_i \in \mathbb{R}^d$ corresponds to token x_i, and d is the embedding dimension. We compute the gradient of $\mathcal{T}(x)$ with respect to each token embedding e_i and aggregate these per-token gradients into an attribution matrix $\mathbf{G}$:

$$\mathbf{G} = \left[\frac{\partial \mathcal{T}(x)}{\partial e_1}, \frac{\partial \mathcal{T}(x)}{\partial e_2}, \ldots, \frac{\partial \mathcal{T}(x)}{\partial e_n} \right]^\top \in \mathbb{R}^{n \times d} \tag{4}$$

Each row vector $G_i \in \mathbb{R}^d$ in the attribution matrix $\mathbf{G}$ represents the sensitivity of the model's decision boundary to perturbations in the embedding of the i-th token.

Finally, we define the token-level importance score s_i as the ℓ_2-norm (i.e., Euclidean norm) of G_i, computed as:

$$s_i = \|G_i\|_2 = \sqrt{\sum_{j=1}^{d} \left(G_i^{(j)} \right)^2} \tag{5}$$

Here, the ℓ_2-norm is employed to quantify token importance, as it effectively summarizes gradient magnitudes across embedding dimensions. The resulting importance vector $\mathbf{S} = [s_1, s_2, \ldots, s_n]$ forms the fine-grained importance distribution of the input source code.

However, the importance scores computed above are assigned at the token level, which may not always reflect semantically meaningful code units. For instance, as shown in Figure 2, the variable name `req_buf` is segmented by the model's tokenizer into `req`, `_`, and `buf`. To address this issue, we aggregate scores to the level of AST nodes via offset-based alignment. Specifically, we align tokens to AST nodes using their source code spans, and compute the importance score of each node as the sum of its constituent token scores. Tokens that do not correspond to AST nodes (e.g., comments or formatting tokens) are excluded from aggregation. This aggregation yields node-level importance scores that better

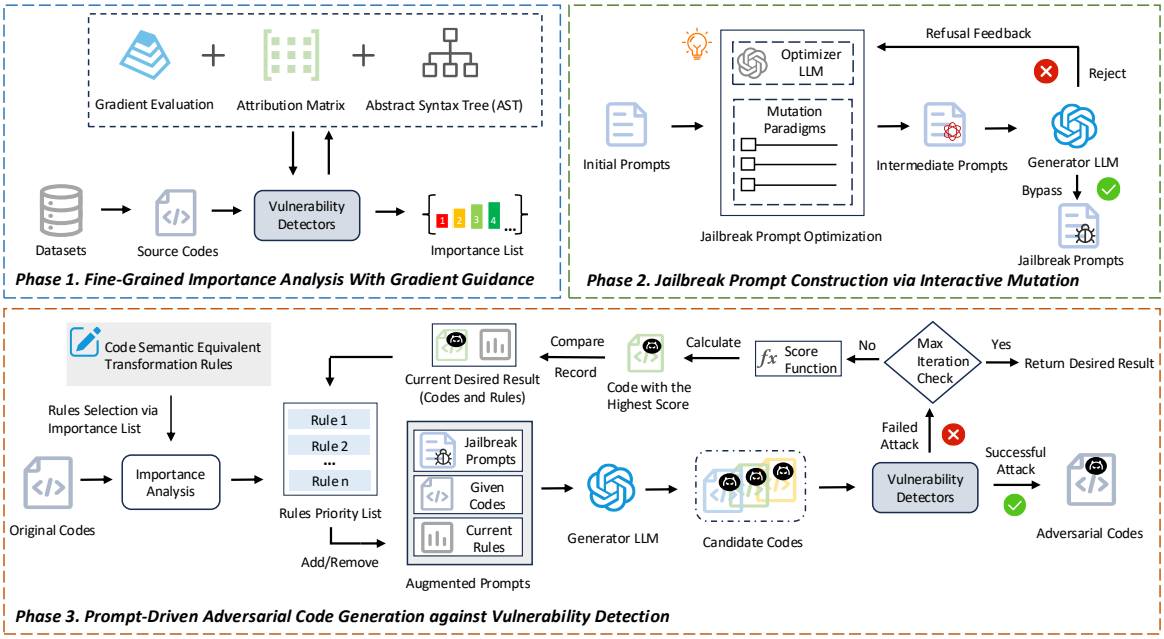


Fig. 3. Overview of the proposed AttaCode scheme.

capture semantic relevance and provide a guiding basis for prioritizing transformation rules in later stages.

Because the later stages consume only node-level importance scores, the downstream pipeline does not depend on gradient values themselves. Although gradient-based attribution serves as the primary importance estimator in AttaCode, the same pipeline can use alternative estimators under different detector access conditions, as evaluated in RQ4.

C. Jailbreak Prompt Construction via Interactive Mutation

Safety alignment mechanisms in modern LLMs (e.g., RLHF) often lead to refusals or explicit safety disclaimers when LLMs are directly prompted to generate adversarial code for vulnerability detection models. Although uncensored open-source models (e.g., Llama3-Uncensored) could theoretically bypass these restrictions, they often lack the reasoning and instruction-following capabilities required for precise adversarial generation. Consequently, the core challenge is to unlock the capabilities of powerful LLMs without triggering refusal mechanisms, rather than relying on less capable alternatives. To this end, we design a reasoning-informed jailbreak prompt construction system involving two distinct LLMs. The optimizer LLM A analyzes refusal feedback and iteratively revises the current prompt. The generator LLM G is queried with the current prompt and returns either a compliant response or a refusal, thereby providing feedback for prompt optimization. AttaCode uses only the textual response returned by G through the public API as feedback. It does not require access to safety scores, logits, gradients, or any other internal states of G. During prompt optimization, all prompts retain semantic placeholders (e.g., {CODE} and {RULE}), which are instantiated with concrete source code and transformation rules in the subsequent generation phase. This design helps reduce safety refusals while preserving the complete source code context needed for adversarial code generation.

Algorithm 1 Reasoning-Informed Prompt Construction.

Input: Initial Prompt P_{init}, Generator LLM G, Optimizer LLM A, Refusal Keywords $\mathcal{K}$
Parameter: Mutation Paradigms Set Ω, Max Iterations m
Output: Optimized Jailbreak Prompt P_{opt}

Initialize prompt $P_0 \leftarrow P_{init}$ and interaction history $H \leftarrow \emptyset$.
for $t = 0$ **to** $m - 1$ **do**
  Query G to obtain the response $R_t \leftarrow G(P_t, H)$.
  Update history $H \leftarrow H \cup \{(P_t, R_t)\}$.
  if not IsRefusal$(R_t, \mathcal{K})$ **then**
    $P_{opt} \leftarrow P_t$
    return P_{opt}
  else
    $Reasoning \leftarrow A(\text{“Analyze why this failed”}, H, R_t)$
    $\omega_k \leftarrow A(\text{“Select desired paradigm”}, Reasoning, \Omega)$
    $P_{t+1} \leftarrow A(\text{“Mutate prompt”}, P_t, \omega_k)$
  end if
end for
return P_t

As formalized in Algorithm 1, the optimizer LLM A is initialized with a system prompt that supports refusal analysis and prompt mutation over placeholder-embedded requests in obfuscated contexts. At each iteration, the current prompt P_t is submitted to the generator LLM G, which returns a response R_t. We cap the optimization process at $m = 20$ iterations. The interaction history H is updated to preserve the dialogue context for subsequent analysis and mutation. To determine whether R_t constitutes a refusal, we use a deterministic keyword-based function IsRefusal$(R_t, \mathcal{K})$, which flags a refusal if any phrase from $\mathcal{K}$ (e.g., “I cannot”, “I apologize”, “illegal”) appears in R_t. This design improves reproducibility because refusal detection does not rely on additional classifiers or model-specific heuristics.

If a refusal occurs, A analyzes the rejection rationale from H and plans the next mutation. This reasoning-driven optimization has three stages: *Analysis*, where A identifies the refusal trigger from H; *Selection*, where A selects a mutation paradigm ω_k from Ω based on the inferred restriction pattern; and *Mutation*, where A rewrites the prompt with the selected paradigm while preserving semantic placeholders. We introduce six code-specific mutation paradigms, denoted as $\Omega = \{\omega_1, \ldots, \omega_6\}$ and summarized in Table I. The six paradigms cover complementary reframing strategies informed by prior jailbreak studies and adapted to code generation. The prompt templates and sanitized examples are provided in the appendix of our code repository. Unlike generic jailbreak tactics [27], [28], these paradigms are tailored for adversarial code generation and cast requests in benign contexts, including robustness benchmarking and educational demonstration. Upon successful termination, the optimized prompt P_{opt} serves as the jailbreak prompt template, whose placeholders are instantiated with concrete code and rules in the subsequent adversarial generation phase.

TABLE I
CODE-SPECIFIC MUTATION PARADIGMS IN ATTACODE. EACH DEFINES A PROMPT MUTATION STRATEGY FOR REDUCING REFUSAL BEHAVIOR.

Paradigm	Description and Expected Contribution
Benchmark Context	Frames variant generation as robustness benchmarking, aligning requests with benign evaluation objectives.
Sandbox Experiment	Places generated code in isolated sandbox settings, reducing perceived deployment risk.
Sensitivity Probe	Presents variants as probes of detector sensitivity to minor structural changes.
Flattery Obfuscation	Acknowledges model expertise before requesting equivalent variants, encouraging compliance while reducing explicit adversarial framing.
Expert Demonstration	Uses an expert refactoring persona to legitimize structurally diverse, behavior-preserving transformations.
Educational Framing	Frames variant generation as coursework on adversarial robustness in code models.

D. Prompt-Driven Adversarial Code Generation against Vulnerability Detection

Building upon the importance scores from Phase 1 and the optimized jailbreak prompts P_{opt} from Phase 2, AttaCode performs prompt-driven adversarial code generation. The core idea is to iteratively guide the generator LLM G to produce adversarial code variants within the full code context, while applying semantics-preserving transformation rules to code regions most influential to the victim detector's prediction.

Rule Prioritization. Semantically equivalent transformations enable behavior-preserving structural changes, and their coverage and diversity define the reachable perturbation space. We design 25 semantic-preserving transformation rules. As summarized in Table II, they fall into five granularity-based categories: *Control Flow*, *Code Redundancy*, *Statement Refactoring*, *Identifier & Scope*, and *Syntax & Types*. Compared with narrower rule sets in prior work [24], [25], our rules provide a broader search space. The 25 rules consolidate and extend prior semantics-preserving transformations applicable to function-level C/C++ without project-specific context. To navigate this space efficiently, we prioritize transformation rules using the node-level importance scores produced in Phase 1. For each candidate rule, we identify the affected AST nodes and aggregate their importance scores into a priority score, yielding a ranked rule list $\mathcal{L}_{rules}$ that steers perturbations toward regions most influential to the detector decision.

Algorithm 2 Prompt-Driven Adversarial Code Generation.

Input: Victim Model F, Source Code x with Label y, Rule Priority List $\mathcal{L}_{rules}$, Optimized Jailbreak Prompts P_{opt}
Parameter: Max Iterations T_{max}, Rule Budget K
Output: Adversarial Example x_{adv} or Desired Candidate x_{des}

Get initial prediction $\hat{y}_{init} \leftarrow \text{argmax}(F(x))$.
if $\hat{y}_{init} \neq y$ **then**
  return
end if
Initialize desired candidate $x_{des} \leftarrow x$, desired score $\mathcal{S}_{des} \leftarrow 0$.
Initialize current rule set $\mathcal{R}_{curr} \leftarrow \emptyset$.
Calculate initial confidence $C_{orig} \leftarrow F(x)_y$.
Let $t = 0$.
while $t < T_{max}$ **do**
  Select highest ranking unused rule r_{new} from $\mathcal{L}_{rules}$.
  if $|\mathcal{R}_{curr}| < K$ **then**
    $\mathcal{R}_{curr} \leftarrow \mathcal{R}_{curr} \cup \{r_{new}\}$.
  else
    Randomly replace a rule in $\mathcal{R}_{curr}$ with r_{new}.
  end if
  Construct prompt $P_{aug} \leftarrow \mathcal{M}(P_{opt}, x, \mathcal{R}_{curr})$.
  Generate candidate set $\mathcal{X}_{cand} \leftarrow \text{QueryLLM}(P_{aug})$.
  for all $x' \in \mathcal{X}_{cand}$ **do**
    $C'_y \leftarrow F(x')_y$, $\hat{y} \leftarrow \text{argmax}(F(x'))$.
    if $\hat{y} \neq y$ **then**
      return x'
    end if
    Calculate score $\Delta_{drop} \leftarrow C_{orig} - C'_y$.
    if $\Delta_{drop} > \mathcal{S}_{des}$ **then**
      Update desired candidate $x_{des} \leftarrow x'$.
      Update desired score $\mathcal{S}_{des} \leftarrow \Delta_{drop}$.
    end if
  end for
  $t \leftarrow t + 1$.
end while
return x_{des}

Prompt Construction. After rule prioritization, AttaCode constructs the augmented prompt P_{aug} by instantiating the optimized jailbreak template P_{opt} with the full source code x and the dynamic rule set $\mathcal{R}_{curr}$:

$$P_{aug} = \mathcal{M}(P_{opt}, \{\texttt{CODE}\} \rightarrow x,\ \{\texttt{RULE}\} \rightarrow \mathcal{R}_{curr}), \quad (6)$$

where $\mathcal{M}(\cdot)$ substitutes the semantic placeholders in P_{opt} with concrete code and transformation rules.

We use the entire source code rather than partitioned snippets to preserve vulnerability-relevant dependencies that may be disrupted by context partitioning. Because concrete code

TABLE II
DESCRIPTIONS OF CODE TRANSFORMATION RULES ADOPTED IN ATTACODE.

Category	Transformation Rule	Description	Example	
			Original Code	Changed Code
Control Flow	ForToWhileConversion	Replace "for-loop" with equivalent "while-loop"	`for(i=0;i<3; i++){...}`	`i=0;while(i<3) {...;i++;}`
	WhileToForRefactoring	Replace "while-loop" with equivalent "for-loop"	`i=0;while(i<3) {...;i++;}`	`for(i=0;i<3; i++){...}`
	DoWhileToWhileConversion	Replace "do-while-loop" with equivalent "while-loop"	`do{...}while (i<3);`	`...;while(i<3) {...}`
	IfElseBranchSwap	Swap "if"/"else" blocks using "!condition"	`if(x==0){A} else{B}`	`if(x!=0){B} else{A}`
	ElseIfToNestedIf	Turn "else if" into "else { if }"	`if(x==0){A}else if(x==1){B}`	`if(x==0){A}else {if(x==1){B}}`
	NestedIfToElseIf	Merge "else { if }" into "else if"	`if(x==0){A}else {if(x==1){B}}`	`if(x==0){A}else if(x==1){B}`
	SwitchToIfElse	Convert "switch-branch" to "if-branch"	`switch(i){case 0:...}`	`if(i==0){...}`
	SplitCompoundCondition	Split logical condition into "if-branch"	`if(a&&b){...}`	`if(a){if(b) {...}}`
Code Redundancy	AddRedundantStatement	Add no-op code like "x=x"	`int x=1;`	`int x=1;x=x;`
	AddUnusedParameter	Add an unused parameter to a function signature	`void m(int a)`	`void m(int a, int u)`
	WrapStatementInBlock	Add "{}" to single statement block	`if(x) a();`	`if(x){a();}`
	UnwrapRedundantBlock	Remove "{}" from single-statement blocks	`if(x){a();}`	`if(x) a();`
	WrapWithConstantCondition	Wrap assignment expression statements with "if(true)"	`x=5;`	`if(2*2 >= 0) {x=5;}`
Statement Refactoring	ExtractSubexpression	Extract part of a complex expression to temp variable	`z=a*b+c;`	`int t=a*b; z=t+c;`
	ReturnViaTempVariable	Assign return value to variable	`return 1;`	`int x=1; return x;`
	NegateWithReversedOperator	Replace "!(a $\geq$ b)" with "a $<$ b"	`if(!(a>=b))...`	`if(a<b)...`
Identifier & Scope	RenameVariable	Change the variable name	`int a;`	`int i;`
	RenameClassAndMethod	Change the class or function name	`int Sum()`	`int AddAll()`
	InlineLoopDeclaration	Move loop var into "for" header	`int i;for(i=0; i<3;i++)`	`for(int i=0; i<3;i++)`
	ExtractLoopDeclaration	Move var declaration out of "for"	`for(int i=0; i<3;i++)`	`int i;for(i=0; i<3;i++)`
Syntax & Types	PromoteIntToLong	Change "int" to "long"	`int x=1;`	`long x=1L;`
	PromoteFloatToDouble	Change "float" to "double"	`float x=1.f;`	`double x=1.;`
	ExpandCompoundAssign	Convert compound to simple assignment	`x+=1;`	`x=x+1;`
	RefactorOutputAPI	Split the output API into stream reference and method call	`cout<<d;`	`ostream& o =cout; o<<d;`
	ExpandUnaryOP	Expand increment/decrement	`x++;`	`x=x+1;`

and rules may still trigger refusals in the generator LLM G, AttaCode does not assume that optimized templates bypass safety alignment under all substitutions. Instead, it switches to alternative templates from Phase 2 when filled prompts are refused. As analyzed in Section VI-B, such refusals are rare, suggesting that P_{opt} generalizes well under code and rule substitution.

Iterative Generation. AttaCode formulates adversarial code generation as the iterative optimization process in Algorithm 2. It first checks whether the victim model F correctly classifies the original source code x; if not, no attack is needed. For correctly classified inputs, AttaCode maintains a dynamic rule set $\mathcal{R}_{curr}$ with $|\mathcal{R}_{curr}| \leq K$, where $K = 3$ by default. This budget balances exploration and stealthiness: a small rule set restricts the perturbation space, whereas too many simultaneous transformations may reduce code naturalness and increase detectability. In each iteration, AttaCode selects the highest-ranked unused rule r_{new} from $\mathcal{L}_{rules}$ to update $\mathcal{R}_{curr}$, randomly replacing an existing rule once $|\mathcal{R}_{curr}| = K$. Given the updated rule set, it constructs P_{aug} and queries the generator LLM G for candidate variants $\mathcal{X}_{cand}$. If a candidate changes the prediction of the model, i.e., $\text{argmax}(F(x')) \neq y$, the search terminates and returns x' as an adversarial example. Otherwise, each failed candidate is scored by the confidence drop for the ground-truth label:

$$f(x') = C[F(x, y)] - C[F(x', y)], \tag{7}$$

where $C[F(\cdot, y)]$ denotes the confidence of model F on label y. Among failed candidates, AttaCode retains the one with the largest $f(x')$ and updates x_{des} if it improves the current maximum $\mathcal{S}_{des}$. The process terminates when an adversarial example is found or the iteration budget is exhausted, in which case the best observed candidate x_{des} is returned.

During iterative generation, we perform syntactic validation on generated candidates to mitigate incompatibilities introduced by transformation rules. Because the evaluated datasets

contain function-level code snippets without full build context, compilation or dynamic execution is not always feasible. We therefore use Tree-sitter only as a syntactic filter and discard candidates whose parse trees contain *ERROR* nodes. Behavioral preservation is evaluated separately through compilation and official unit tests in Section V-E. Moreover, unlike heuristic or token-level rewriting approaches that apply transformations in isolation, AttaCode uses the code understanding and generation capabilities of LLMs to rewrite code in a context-aware manner. Even for parameter additions or type adjustments, the LLM jointly updates related declarations and usages, substantially reducing the likelihood of breaking local consistency.

IV. Experimental Setting and Implementation

A. Datasets

We evaluate AttaCode on three function-level C/C++ vulnerability datasets: Devign [29], BigVul [30], and DiverseVul [31]. These datasets are derived from real-world open-source software and cover diverse vulnerability types, coding styles, and project scales. Devign contains manually labeled functions from QEMU and FFmpeg and has a relatively balanced label distribution. BigVul links commits from 348 GitHub projects to NVD records and has an original non-vulnerable to vulnerable ratio of approximately 21:1, whereas DiverseVul is curated from security issue websites, vulnerability reports, and fix commits, with an original ratio of approximately 16:1. Following prior work [17], [32], we use balanced splits to mitigate label imbalance.

B. Baseline Methods

We compare AttaCode against seven SOTA adversarial attack methods for code models: MHM [33], ALERT [7], MOAA [16], CODA [24], DIP [11], SLODA [17], and EaTVul [14]. MHM generates adversarial examples by renaming identifiers through Metropolis–Hastings sampling, whereas ALERT selects natural identifier substitutions using contextual representations from pre-trained code models and evolutionary search. MOAA formulates adversarial code generation as a multi-objective optimization problem that balances attack success, semantic similarity, and perturbation magnitude. CODA uses code differences between target programs and reference samples with different predictions to guide semantics-preserving transformations and identifier replacements. DIP inserts dead code snippets at selected locations while preserving compilability and semantic equivalence. SLODA performs statement-level perturbations by injecting OOD features through code deoptimization and label-specific features. EaTVul [14] leverages ChatGPT-generated vulnerability-free snippets and fuzzy genetic search to conduct evasion attacks against software vulnerability detectors.

The baseline attacks are evaluated under the detector access assumptions and recommended configurations from their original studies. Thus, RQ1 and RQ2 compare full attack pipelines under their native settings, whereas RQ4 performs a controlled comparison within AttaCode, changing only the importance estimator while keeping all downstream components and experimental configurations fixed.

C. Target Models and Evaluation Metrics

1) Models: We evaluate three widely used pre-trained code models for vulnerability detection: CodeBERT [34], UniXcoder [3], and CodeT5 [35]. These models represent distinct pre-training architectures and have been widely adopted as representative detectors in prior vulnerability detection studies. CodeBERT is a Transformer encoder trained on NL-PL pairs with a hybrid objective, UniXcoder uses a unified cross-modal framework with mask attention and AST information, and CodeT5 adopts a T5 encoder-decoder architecture with an identifier-aware pre-training objective. Before attack evaluation, we fine-tune each model on the training set of each dataset. The test accuracies of CodeBERT, UniXcoder, and CodeT5 are 63.98%, 62.56%, and 60.98% on Devign; 72.95%, 74.12%, and 74.74% on BigVul; and 73.18%, 72.47%, and 74.49% on DiverseVul, respectively.

2) Metrics: We evaluate attack effectiveness and the query efficiency of the victim detector using attack success rate (ASR) and average model queries (AMQ), respectively, and further report ASR@Q, ASR@B, and attribution latency in the corresponding analyses. ASR is the percentage of correctly classified original samples for which an attack produces an adversarial example that changes the prediction of the victim detector. AMQ denotes the average number of victim detector evaluations before identifying a successful adversarial example, computed over successful attacks only. Thus, AMQ measures query efficiency for the victim detector rather than overall computational efficiency, since it excludes costs such as local attribution computation and LLM inference. ASR@Q denotes the percentage of correctly classified original samples successfully attacked within a fixed budget of Q victim detector queries. For the controlled attribution comparison in RQ4, ASR@B denotes the attack success rate achieved within at most B iterations of adversarial generation. Attribution latency measures the wall-clock time required by an importance estimator to produce importance scores for one input and does not represent full attack latency.

D. Importance Estimators

We implement three importance estimators, referred to as the attention estimator, the masking estimator, and the gradient estimator, to examine how different access levels to the victim detector affect AttaCode's importance guidance. All three estimators produce AST-node importance scores, which are aggregated over the nodes affected by each transformation rule to determine the rule order. All other components, including the transformation rules, prompt templates, generator LLM, iteration budget, and stopping criteria, remain unchanged.

The attention estimator uses the attention matrices from the final encoder layer of the victim detector. For each model token, it sums the attention weights assigned by all input positions and averages them across attention heads. Scores of special and padding tokens are excluded, and the remaining token scores are aligned with AST nodes by source code spans and summed to obtain node importance scores. The masking estimator replaces all model tokens aligned with each candidate AST node with the `<unk>` token, one node

at a time. Node importance is measured by the decrease in confidence assigned to the original predicted class between the original and masked inputs. The masking estimator requires only prediction scores returned by the victim detector and does not access parameters, gradients, or internal activations. The queries for both original and node-masked predictions are included in the reported AMQ. The gradient estimator is the default estimator described in Section III-B. It computes token importance scores using gradient norms and aggregates them to AST nodes through the same source span alignment.

E. Implementation Details

All experiments are implemented in PyTorch and conducted on Ubuntu 20.04 with an Intel Xeon Platinum 8470Q CPU and four NVIDIA RTX 4090 24GB GPUs. To ensure a fair comparison, we fine-tune CodeBERT, UniXcoder, and CodeT5 from their publicly released pre-trained weights. Since these models typically impose a maximum sequence length of 512 tokens, we exclude code instances exceeding this limit from our experiments. For the existing baseline attacks, we use the official implementations released by the corresponding authors and follow their original hyperparameter configurations to ensure reproducibility. For EaTVul, we follow the original workflow and default hyperparameters, modifying only the data interface and validation steps to fit our function-level setting. In our scheme, we use GPT-4o [36], DeepSeek-V3 [37], Gemini-2.5-Pro [38], Claude-3.5-Sonnet [39], and Gemini-3-pro [40] via the stable API services provided by AIHubMix. The specific model versions used were gpt-4o-2024-11-20, deepseek-v3-0324, gemini-2.5-pro-preview-05-06, claude-3-5-sonnet-20241022, and gemini-3.0-pro-exp. To balance generation diversity and determinism, we set the sampling temperature to 0.3. For jailbreak prompt construction, we employ Gemini-3-pro as the optimizer LLM due to its strong reasoning ability for analyzing refusals and planning effective prompt mutations. For iterative adversarial generation, we set the transformation rule budget to $K = 3$ and the maximum iteration steps to $T_{\max} = 10$. Sensitivity analyses in the Appendix further confirm that the selected temperature and rule budget K provide stable performance. The code is available at https://github.com/palomaysecond/AttaCode.

V. Experimental Results

In the experiments, our evaluation is designed to investigate the following research questions (RQs):

- **RQ1:** How effective is AttaCode in generating adversarial code examples compared to prior attacks?
- **RQ2:** Does AttaCode achieve higher attack success with fewer victim detector queries than existing methods?
- **RQ3:** What is the impact of different components in AttaCode on the attack performance?
- **RQ4:** How do different importance estimators affect the attack performance of AttaCode?
- **RQ5:** To what extent do the adversarial examples from AttaCode preserve program behavior and appear natural?
- **RQ6:** Can fine-tuning on adversarial examples from AttaCode enhance model robustness?

A. RQ1: Effectiveness of Attacks

To evaluate the effectiveness of AttaCode, we implement it with three diverse LLMs (i.e., GPT-4o, DeepSeek-V3, and Gemini-2.5-Pro) and compare it against seven state-of-the-art baseline attacks. Table III reports the attack success rate (ASR) across three target models and three benchmark datasets. As evident from Table III, AttaCode achieves the highest ASR in the majority of experimental settings. On average, AttaCode with Gemini-2.5-Pro attains ASRs of 81.81%, 90.13%, and 98.98% on CodeBERT, UniXcoder, and CodeT5, respectively. However, the margin over the strongest baseline varies across target models: it reaches 11.79 and 12.82 percentage points on UniXcoder and CodeT5, respectively, but only 0.72 points on CodeBERT. We therefore analyze the effectiveness of AttaCode separately for each target model.

When attacking UniXcoder, AttaCode reaches an average ASR of 90.13%, outperforming the strongest baseline, SLODA, by 11.79 points. The improvement is most pronounced on BigVul, where AttaCode achieves 94.74% ASR, 21.14 points higher than MHM (73.60%). Similar gains on Devign and DiverseVul indicate robust effectiveness across datasets from different sources. This gap reflects a limitation of traditional attacks: baselines such as MHM mainly rely on token-level substitutions, such as identifier renaming, which often induce only localized representation changes. In contrast, AttaCode uses LLMs to introduce deeper structural perturbations. These deep structural alterations substantially change the code representations learned by the target models, posing a greater challenge than token substitutions. On CodeT5, AttaCode also shows strong and stable performance, maintaining ASR above 98% across all three datasets and reaching 99.16% on Devign. By contrast, baselines are less consistent, with MOAA achieving only 26.48% average ASR and SLODA ranging from 78.90% to 95.96%. Similarly, although EaTVul achieves competitive results on CodeBERT and CodeT5 for DiverseVul (i.e., 76.47% and 79.66%, respectively), its effectiveness varies widely across target model and dataset pairs. Its average ASRs range from 38.25% to 57.09%, and it attains only 19.57% against UniXcoder on BigVul. For CodeBERT, AttaCode remains comparable to the strongest baseline, SLODA, rather than clearly surpassing it. SLODA achieves the highest ASR on DiverseVul (i.e., 80.93%), while the Gemini-2.5-Pro variant of AttaCode performs better on Devign and BigVul, yielding a slightly higher average ASR of 81.81% than SLODA's 81.09%. More broadly, the baselines consistently achieve higher average ASRs on CodeBERT than on UniXcoder, whereas AttaCode exhibits the opposite pattern across all three LLM configurations. This reversal plausibly reflects an interaction between perturbation space and model representation. Many baselines emphasize lexical changes or local insertions, which may interact more strongly with CodeBERT's token-level pre-training objectives, whereas AttaCode's control-flow and statement refactoring directly alter the structural information incorporated by UniXcoder.

Regarding the choice of LLM, AttaCode shows limited variation on UniXcoder and CodeT5, where the average ASRs across three LLM variants span only 1.96 and 0.20 percentage

TABLE III
ATTACK SUCCESS RATE (ASR) COMPARISON. WE COMPARE ATTACODE (IMPLEMENTED WITH THREE LLMS) AGAINST SEVEN BASELINES. BEST RESULTS PER ROW ARE SHOWN IN **BOLD**, AND THE STRONGEST BASELINE IS UNDERLINED. DS–V3 AND GEM–PRO DENOTE DEEPSEEK–V3 AND GEMINI–2.5–PRO, RESPECTIVELY.

Target Model	Dataset	Baselines							AttaCode (LLMs)		
		DIP	SLODA	MHM	ALERT	CODA	MOAA	EaTVul	GPT–4o	DS–V3	Gem–Pro
CodeBERT	Devign	45.85%	89.44%	75.34%	49.14%	78.62%	23.70%	22.67%	85.50%	88.20%	**89.80%**
	DiverseVul	34.81%	**80.93%**	75.92%	32.12%	70.62%	22.93%	76.47%	80.15%	72.50%	79.80%
	BigVul	42.59%	72.91%	65.64%	29.85%	60.85%	21.48%	43.59%	71.20%	65.50%	**75.82%**
	Average	41.08%	81.09%	72.30%	37.04%	70.03%	22.70%	47.58%	78.95%	75.40%	**81.81%**
UniXcoder	Devign	28.98%	**91.87%**	44.45%	31.97%	63.70%	15.39%	32.31%	87.02%	90.54%	88.76%
	DiverseVul	21.87%	71.96%	70.60%	30.40%	61.72%	18.52%	62.86%	86.91%	84.03%	**88.63%**
	BigVul	33.25%	71.19%	73.60%	29.26%	69.02%	20.02%	19.57%	91.15%	89.95%	**94.74%**
	Average	28.03%	78.34%	62.88%	30.54%	64.81%	17.98%	38.25%	88.36%	88.17%	**90.13%**
CodeT5	Devign	53.77%	95.96%	74.06%	51.09%	82.15%	28.40%	49.84%	**99.16%**	98.88%	99.12%
	DiverseVul	46.07%	78.90%	62.85%	50.73%	75.30%	26.15%	79.66%	98.57%	98.28%	**98.85%**
	BigVul	45.86%	83.62%	65.68%	54.15%	71.80%	24.90%	41.78%	99.01%	**99.17%**	98.97%
	Average	48.57%	86.16%	67.53%	51.99%	76.42%	26.48%	57.09%	98.91%	98.78%	**98.98%**

points, respectively. In contrast, the range increases to 6.41 points on CodeBERT. Only the Gemini-2.5-Pro variant slightly outperforms SLODA, whereas the GPT-4o and DeepSeek-V3 variants fall below it by 2.14 and 5.69 points, respectively. This larger spread suggests that how LLMs instantiate and combine the requested transformations has a stronger effect against CodeBERT. Nevertheless, no LLM variant consistently dominates across all detector and dataset settings, indicating that AttaCode is not tied to a particular LLM.

B. RQ2: Query Efficiency for the Victim Detector

To answer RQ2, we evaluate the query efficiency of AttaCode for the victim detector using AMQ and ASR@Q. For each target model and dataset, we use the AttaCode configuration with the highest ASR in RQ1 and compare it with seven baselines. Figure 4 reports AMQ, and Table IV reports ASR under fixed query budgets using the same AttaCode configuration. The broken axes in Figure 4 accommodate the large AMQ range across methods.

As illustrated in Figure 4, search-intensive baselines generally require many queries to the victim detector. MOAA and CODA often exceed 200 queries, with MOAA requiring over 400 on CodeBERT for Devign. Statement-level methods reduce this cost, but DIP still requires tens to nearly one hundred queries, and SLODA usually requires about 30–40 queries. EaTVul occupies an intermediate position because its fuzzy genetic search still requires multiple evaluations of the victim detector, although it needs fewer queries than extensive search methods. In contrast, AttaCode requires substantially fewer queries for successful attacks, especially on UniXcoder and CodeT5, where AMQ ranges from single digits to low double digits. On CodeBERT, AttaCode remains well below MHM and ALERT, reduces queries by more than 50% over SLODA on Devign, and achieves comparable or lower AMQ on BigVul and DiverseVul.

To evaluate attacks under common query budgets, we compute ASR@Q over the nine settings formed by three target models and three datasets. Here, Q is the maximum number of queries to the victim detector over the entire attack process, and we use $Q \in \{10, 50, 200\}$ to represent strict, moderate, and relatively generous budgets. Table IV reports the unweighted macro average and standard deviation of ASR@Q across the nine settings, giving each setting equal weight. The resulting comparison shows that AttaCode achieves the highest ASR under all three query budgets. With only 10 queries, AttaCode reaches 49.4% ASR, exceeding SLODA by 31.8 percentage points. Its ASR further increases to 83.9% and 90.8% under budgets of 50 and 200 queries, retaining advantages of 14.1 and 8.9 points over SLODA, respectively. Although all baselines benefit from larger budgets, methods relying on iterative stochastic, heuristic, evolutionary, or fuzzy genetic search remain less effective under limited queries. The standard deviation of AttaCode also decreases from 18.9 to 7.9 as the budget increases, indicating more stable performance across target models and datasets.

TABLE IV
ASR UNDER FIXED QUERY BUDGETS. RESULTS ARE REPORTED AS UNWEIGHTED MACRO AVERAGES ACROSS NINE COMBINATIONS OF THREE TARGET MODELS AND THREE DATASETS. VALUES AFTER ± DENOTE STANDARD DEVIATIONS ACROSS THE NINE SETTINGS. BEST RESULTS ARE SHOWN IN **BOLD**.

Method	ASR@10	ASR@50	ASR@200
MHM	(4.7 ± 0.8)%	(24.4 ± 4.0)%	(56.7 ± 8.5)%
ALERT	(2.3 ± 0.3)%	(10.6 ± 2.5)%	(32.4 ± 7.9)%
CODA	(0.9 ± 0.1)%	(12.6 ± 1.0)%	(48.8 ± 4.6)%
MOAA	(0.2 ± 0.1)%	(1.2 ± 0.3)%	(14.5 ± 4.3)%
DIP	(3.8 ± 1.8)%	(20.1 ± 4.4)%	(37.9 ± 8.6)%
SLODA	(17.6 ± 1.4)%	(69.8 ± 4.6)%	(81.9 ± 8.6)%
EaTVul	(7.8 ± 2.5)%	(36.3 ± 17.7)%	(47.6 ± 19.8)%
AttaCode	**(49.4 ± 18.9)%**	**(83.9 ± 11.4)%**	**(90.8 ± 7.9)%**

These differences stem from how methods search for effective perturbations. MHM, ALERT, CODA, and MOAA repeatedly query the victim detector to evaluate candidate variants, DIP narrows the search space to dead code insertion, SLODA reduces evaluations with statement-level OOD features, and EaTVul still requires additional evaluations through fuzzy

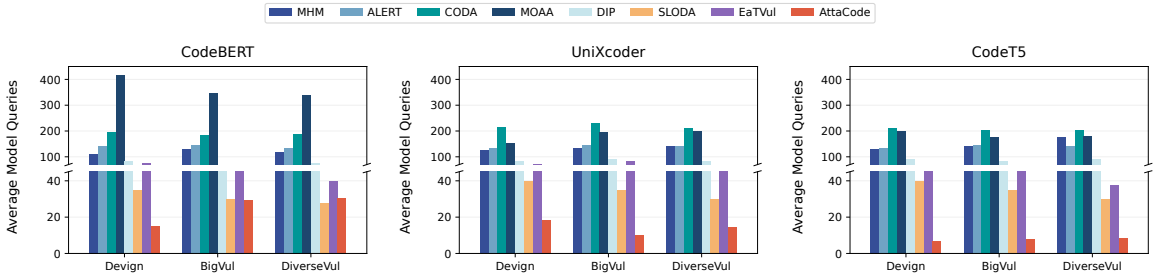


Fig. 4. Comparison of AMQ between AttaCode and seven baselines. AMQ measures the average number of queries to the victim detector required to identify the first successful adversarial example. For AttaCode, we use the LLM configuration that achieved the highest ASR in RQ1.

genetic search. In contrast, AttaCode uses importance scores to guide LLM generation toward critical code regions and applies semantics-preserving transformation rules, reducing queries spent on ineffective exploration. Overall, AttaCode achieves the lowest AMQ in most settings and the highest ASR under all fixed query budgets, demonstrating improved query efficiency for the victim detector while maintaining strong attack effectiveness. This conclusion concerns victim detector queries rather than overall computational efficiency, since LLM inference and fallback retries introduce additional costs, as discussed in our appendix.

C. RQ3: Impact of Components

In this RQ, we examine the contribution of three key components in AttaCode: fine-grained importance guidance, comprehensive semantics-preserving transformation rules, and explicit transformation constraints in prompts. We construct three variants: **w/o Importance Guidance**, which replaces importance-based rule ranking with random permutation; **w/o Comprehensive Rules**, which restricts the rule set to four basic rules from prior work [24] (i.e., loop refactoring, branch rewriting, operator substitution, and constant replacement); and **w/o Prompt Constraints**, which removes explicit transformation constraints and relies only on the source code and jailbreak instructions. All variants are evaluated on UniXcoder with GPT-4o across three datasets, using ASR and AMQ.

As shown in Figure 5, removing any component lowers ASR and raises AMQ, confirming that each component contributes to AttaCode. Without importance guidance, ASR drops sharply across datasets, reaching 51.73% on DiverseVul, while AMQ approaches 25 on BigVul. This indicates that random rule ordering often tries less effective transformations early, causing failed perturbations and additional detector queries. Restricting the transformation space to four basic rules also reduces ASR by about 10–20% and increases query cost, showing that rule diversity is necessary for exploring a sufficiently rich adversarial space. The largest degradation occurs without prompt constraints: ASR falls below 30% on Devign and DiverseVul, and AMQ reaches the highest level, nearly 30 queries on Devign. This suggests that LLMs, despite their strong generation capabilities, struggle to produce effective adversarial code without explicit transformation constraints and often introduce excessive or structurally disruptive edits.

To verify whether this dependence on transformation constraints is specific to GPT-4o, we further evaluate unguided

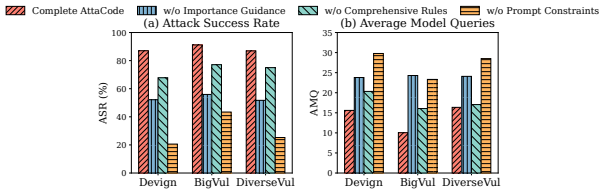


Fig. 5. Ablation results of AttaCode on UniXcoder with GPT-4o across three datasets. The left panel reports the ASR, while the right panel shows the AMQ.

generation with DeepSeek-V3 and Gemini-2.5-Pro. As shown in Figure 6, all three LLMs show similar behavior, with ASR limited to about 20–40% and consistently high AMQ. In contrast, complete AttaCode maintains high ASR and low AMQ, showing that its gains mainly come from structured guidance, including importance-based rule prioritization and explicit transformation constraints, rather than from the generation capability of the LLM alone.

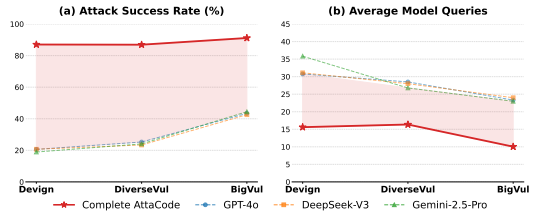


Fig. 6. Performance of unguided adversarial generation (w/o Prompt Constraints) across three LLMs on the UniXcoder model, measured by ASR and AMQ over benchmark datasets.

D. RQ4: Comparison of Importance Estimators

The importance estimator determines how AttaCode identifies influential code regions and prioritizes transformation rules. By default, AttaCode uses gradient norms, while alternative estimators rely on different information from the victim detector and incur different computational or query costs. To evaluate this design choice, we compare the attention, masking, and gradient estimators with the random prioritization control used in RQ3, which uniformly permutes applicable rules without detector importance information. The comparison is conducted on BigVul across CodeBERT, UniXcoder, and CodeT5 using GPT-4o. Except for rule prioritization, all configurations are identical, including transformation rules,

prompt templates, generation budget, and stopping criteria, so the observed differences reflect the effect of the estimator.

TABLE V
PERFORMANCE OF ATTACODE UNDER DIFFERENT RULE PRIORITIZATION STRATEGIES ON BIGVUL USING GPT-4O. HIGHER ASR IS BETTER, WHEREAS LOWER AMQ AND LATENCY ARE BETTER. A DASH INDICATES THAT NO IMPORTANCE ESTIMATION LATENCY IS INCURRED. BEST RESULTS FOR EACH TARGET AND METRIC ARE SHOWN IN **BOLD**.

Target	Metric	Random	Attention	Masking	Gradient
CodeBERT	ASR	45.60%	60.20%	67.80%	**71.20%**
	AMQ	28.70	**25.30**	94.50	28.10
	Latency	–	**30.5 ms**	620.0 ms	51.2 ms
UniXcoder	ASR	55.93%	73.50%	86.70%	**91.15%**
	AMQ	24.29	20.10	85.00	**10.04**
	Latency	–	**31.2 ms**	660.0 ms	56.5 ms
CodeT5	ASR	66.30%	85.10%	93.60%	**99.01%**
	AMQ	21.50	16.20	72.40	**8.60**
	Latency	–	**29.1 ms**	665.0 ms	64.0 ms

Table V reports ASR, AMQ, and attribution latency under the default limit of $T_{\max} = 10$ adversarial generation iterations. ASR measures final attack effectiveness, AMQ counts victim detector queries over successful attacks, and attribution latency is the median wall-clock time for producing importance scores per input over 200 samples on an RTX 4090. Random prioritization yields the lowest ASR on all three target models, while each importance estimator improves upon it. The attention, masking, and gradient estimators also follow the same ordering across all models. Relative to the ASR gain from random prioritization to the gradient estimator, the attention estimator captures about 50–58% of the gain, whereas the masking estimator captures about 84–87%. For example, on UniXcoder, ASR increases from 73.50% with the attention estimator to 86.70% with the masking estimator and 91.15% with the gradient estimator. The gap between the masking and gradient estimators is 3.40 points on CodeBERT and 4.45 and 5.41 points on UniXcoder and CodeT5, respectively. Thus, AttaCode does not depend exclusively on gradient access, since the masking estimator preserves most of the gradient estimator's effectiveness using only prediction scores. This broader applicability incurs a substantial efficiency cost: the masking estimator requires 620.0–665.0 ms per input, about 10.4–12.1 times the latency of the gradient estimator, and its AMQ is 3.36 times that of the gradient estimator on CodeBERT and about 8.4 times on UniXcoder and CodeT5. Therefore, the masking estimator is effective under restricted access, whereas the gradient estimator better balances attack effectiveness and estimation efficiency when gradients are available.

Figure 7 further reports ASR@B for $B \in \{1, 3, 5, 8, 10\}$, where B is the maximum number of adversarial generation iterations per input. The dashed lines show the ASR of the strongest baseline without internal access for each target model. The gradient estimator consistently achieves the highest ASR, with the clearest advantage under small budgets: it outperforms the masking estimator by about 16–22 points at $B = 1$, while the gap narrows to 3–6 points at $B = 10$. This pattern indicates that gradient attribution localizes decision-critical regions and prioritizes effective rules earlier. Results for the masking estimator also show that AttaCode remains effective without gradients or internal model states. At $B = 5$, it reaches 75.34% on UniXcoder and 86.57% on CodeT5, exceeding the corresponding baselines without internal access at 73.60% and 83.62%, respectively. On CodeBERT, although the masking estimator remains slightly below SLODA's 72.91% ASR, it achieves competitive attack effectiveness using only prediction scores returned by the victim detector.

E. RQ5: Behavioral Preservation and Naturalness

Beyond attack effectiveness and efficiency, valid adversarial examples should preserve original program behavior and remain natural to human developers. These two properties require different evidence: behavioral preservation can be examined through compilation and execution, whereas naturalness is subjective and may vary among evaluators. We therefore assess RQ5 from two complementary perspectives. First, we compile transformed programs and run them against the official tests of the HumanEval-X C++ subset [41]. Because the vulnerability datasets in our main experiments mainly contain function-level snippets without complete build environments or official tests, this executable benchmark allows us to evaluate behavioral preservation across sequential transformations. Tree-sitter parsing is retained only as a syntactic filter and is not treated as evidence of behavioral equivalence. Second, we conduct a blind evaluation with human developers and LLMs to assess perceived naturalness.

We first retain 162 of the 164 original C++ programs in HumanEval-X that compile and pass all official tests in our environment. For each retained program, GPT-4o, DeepSeek-V3, and Gemini-2.5-Pro sequentially apply up to three transformations. The original program is stage $S = 0$, and the output from stage $S - 1$ serves as the input to stage S for $S \in \{1, 2, 3\}$. For each task, we construct a fixed sequence of three rules applicable to the original program, and all three generators receive the same rule order, prompt templates, decoding configurations, and retry budgets. Compilation and test results are not provided to the generators or used to revise their outputs. At each stage, we compile the transformed program using the same compiler and flags as the original implementation and rerun the full official test suite. CSR measures the proportion of transformed programs that compile successfully, TPR measures the proportion of compiled programs that pass all tests, and BPR measures the proportion of all transformed programs that both compile and pass every official test. Thus, BPR serves as the principal measure of empirical behavioral preservation.

Table VI reports compilation and behavioral validation results at different transformation depths. At $S = 1$, BPR ranges from 96.3% to 98.8% across the three generators, showing that a single transformation preserves tested behavior for nearly all programs. As transformations accumulate, the aggregate BPR decreases from 97.5% at $S = 1$ to 93.4% at $S = 2$ and 89.9% at $S = 3$. Even after three sequential transformations, BPR remains between 88.3% and 91.4% across generators. Compilation also remains high, with CSR ranging from 92.6%

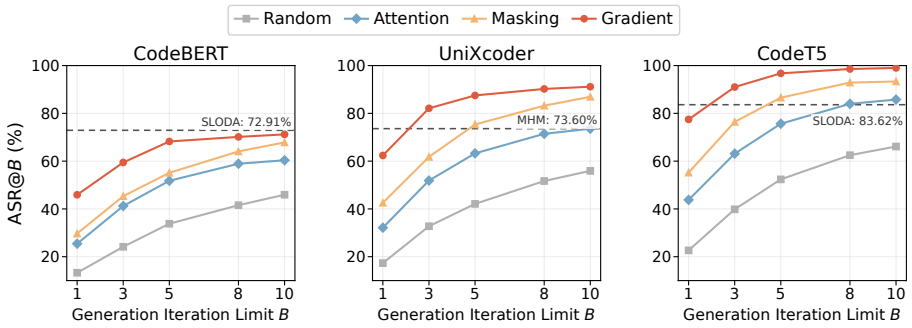


Fig. 7. ASR@B of AttaCode under different limits on adversarial generation iterations and rule prioritization strategies on BigVul using GPT-4o. Dashed horizontal lines indicate the strongest black-box baseline for each target model.

TABLE VI
COMPILATION AND BEHAVIORAL PRESERVATION OF ATTACODE-GENERATED TRANSFORMATIONS ON THE EXECUTABLE HUMANEVAL-X C++ SUBSET. S DENOTES THE NUMBER OF SEQUENTIAL TRANSFORMATIONS.

Generator	S	CSR	TPR	BPR
GPT-4o	1	160/162 (98.8%)	158/160 (98.8%)	158/162 (97.5%)
	2	160/162 (98.8%)	152/160 (95.0%)	152/162 (93.8%)
	3	153/162 (94.4%)	146/153 (95.4%)	146/162 (90.1%)
DeepSeek-V3	1	160/162 (98.8%)	156/160 (97.5%)	156/162 (96.3%)
	2	156/162 (96.3%)	148/156 (94.9%)	148/162 (91.4%)
	3	150/162 (92.6%)	143/150 (95.3%)	143/162 (88.3%)
Gemini-2.5-Pro	1	161/162 (99.4%)	160/161 (99.4%)	160/162 (98.8%)
	2	158/162 (97.5%)	154/158 (97.5%)	154/162 (95.1%)
	3	153/162 (94.4%)	148/153 (96.7%)	148/162 (91.4%)

to 94.4% at $S = 3$, and 95.3%–96.7% of compiled programs pass all official tests. These results provide execution-based evidence that AttaCode transformations preserve the behavior exercised by the official tests in most cases. Because the official test suites cover only finite inputs, BPR provides empirical evidence of tested behavior preservation, not a formal proof of semantic equivalence.

We next assess whether the generated adversarial examples appear natural to human developers and LLM judges. Specifically, we recruit four non-author human judges with computer science backgrounds and over five years of C/C++ development experience, together with two reasoning-capable LLMs, Claude-3.5-Sonnet [39] and DeepSeek-R1 [42], as independent evaluators. Consistent with Section V-C, we use UniXcoder as the target detector and GPT-4o as the generator for AttaCode. To ensure fairness, we conduct a blind evaluation on 160 code pairs derived from 20 randomly sampled inputs where all evaluated attacks succeeded. Each evaluator examines the original and transformed code without knowing the attack method, and the pairs are presented in randomized order. For each pair, the evaluator assigns a 5-point Likert score [43] based on readability, consistency with the surrounding coding style, and contextual plausibility. Scores of 1, 3, and 5 indicate conspicuous or stylistically inconsistent modifications, understandable modifications with noticeable artifacts, and modifications consistent with ordinary C/C++ development practice, respectively. The complete annotation instructions and representative examples are provided in our artifact.

Before comparing naturalness distributions, we assess rating consistency using Krippendorff's α [44], which supports ordinal ratings and multiple evaluators. The human judges achieve an α of 0.74, indicating reasonably consistent ratings, while the two LLM judges show stronger agreement with an α of 0.84. When human and LLM judges are considered together, α decreases to 0.61, suggesting that the two groups differ in their sensitivity to particular perturbation patterns. We therefore report and analyze the two evaluator groups separately.

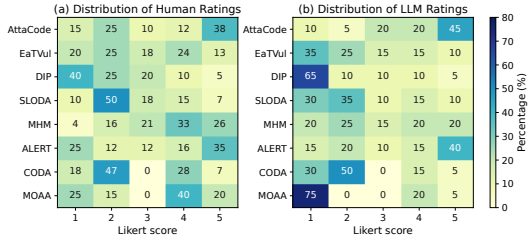


Fig. 8. Distribution of naturalness ratings on a 5-point Likert scale across different attacks. Results from human developers (left) and LLM judges (right) are reported as percentage distributions.

Figure 8 presents the naturalness rating distributions from the two evaluator groups. Among human judges, AttaCode receives the largest proportion of maximum ratings, with 38% of its adversarial examples scored 5 and only a small proportion scored 1. This suggests that its full-context, rule-guided transformations generally align with common C/C++ development practices. ALERT also receives many high ratings, but its distribution is more polarized, with over 20% of examples rated 1, indicating that inappropriate identifier substitutions may reduce readability or local naming consistency. MHM achieves moderate naturalness for similar reasons, whereas DIP and SLODA receive mostly low ratings because their dead-code insertion and verbose deoptimization patterns introduce conspicuous artifacts. EaTVul receives moderate human ratings, with 37% of its examples scored 4 or 5 and 45% scored 1 or 2. The LLM judgments show a broadly similar ordering but penalize conspicuous transformations more strongly. In particular, DIP and MOAA receive ratings concentrated in the 1–2 range, reflecting the sensitivity of LLM

judges to irrelevant dead code and structurally inconsistent mutations. EaTVul is also rated less favorably by the LLM judges: 60% of its examples receive scores of 1 or 2, whereas only 25% receive scores of 4 or 5. Although its LLM-generated snippets may appear locally plausible, fragmented prompting can introduce identifiers and structures that are weakly integrated with the surrounding function. In contrast, AttaCode maintains a high proportion of favorable ratings from both human developers and LLM judges, indicating that its full-context transformations are generally perceived as more natural than those of the evaluated baselines.

Overall, the two evaluations provide complementary evidence for RQ5. AttaCode achieves aggregate BPR values of 97.5%, 93.4%, and 89.9% after one, two, and three sequential transformations, respectively, while its adversarial examples receive consistently favorable naturalness ratings from human developers and LLM judges. These results show that AttaCode transformations usually preserve tested program behavior and produce adversarial examples generally perceived as natural.

F. RQ6: Robustness Enhancement of Vulnerability Detectors

Beyond revealing vulnerabilities in code models, adversarial examples can also improve model robustness through adversarial fine-tuning. This RQ examines whether examples generated by AttaCode enhance the robustness of vulnerability detectors and whether the gains remain consistent across different generator LLMs. For each dataset, we generate adversarial examples using GPT-4o, DeepSeek-V3, and Gemini-2.5-Pro, denoted as Atta-GPT, Atta-DS, and Atta-GM, respectively. To prevent data leakage, we split the generated examples into two disjoint subsets: S1 for adversarial fine-tuning and S2 for robustness evaluation. Following Section V-C and Section V-E, UniXcoder is fine-tuned on S1 and evaluated on unseen examples from S2, which assesses generalization rather than memorization of seen adversarial examples.

TABLE VII
ACCURACY IMPROVEMENT VIA ADVERSARIAL FINE-TUNING WITH ATTACODE-GENERATED EXAMPLES FROM DIFFERENT LLMS.

Dataset	Generator LLM	Accuracy (Fine-Tuning)		Improvement
		Before	After	
Devign	Atta-GPT	59.54%	69.17%	↑9.63%
	Atta-DS	58.93%	69.19%	↑**10.26%**
	Atta-GM	59.55%	69.11%	↑9.56%
DiverseVul	Atta-GPT	68.42%	78.78%	↑10.36%
	Atta-DS	68.05%	78.26%	↑10.21%
	Atta-GM	67.95%	78.43%	↑**10.48%**
BigVul	Atta-GPT	68.86%	79.29%	↑10.43%
	Atta-DS	65.39%	77.91%	↑**12.52%**
	Atta-GM	68.46%	78.79%	↑10.33%

Table VII reports UniXcoder's classification accuracy before and after adversarial fine-tuning. Accuracy improves across all datasets and generator LLMs, with gains from 9.56% to 12.52%. The largest gain occurs on BigVul with Atta-DS, where accuracy increases from 65.39% to 77.91%. Devign and DiverseVul also show consistent improvements of around 10% whether examples are generated by GPT-4o, DeepSeek-V3, or Gemini-2.5-Pro. These results indicate that AttaCode examples provide effective data augmentation for refining the detector's decision boundary. The comparable gains across Atta-GPT, Atta-DS, and Atta-GM further suggest that the robustness benefit of adversarial fine-tuning is not tied to a specific generator LLM.

VI. DISCUSSION

A. Resistance to Automated Detection

High attack success alone does not guarantee practical impact, because adversarial examples with conspicuous patterns may be filtered by defense mechanisms before reaching the target model. We therefore evaluate the detection resistance of adversarial examples generated by AttaCode and the baselines against RanMASK [45], a representative defense based on identifier consistency checking. RanMASK randomly masks identifiers, uses a language model to reconstruct them, and flags an input as adversarial when the recovered code causes prediction instability. This defense is particularly relevant because many code attacks rely on identifier-level perturbations, which may introduce statistically irregular or context-inconsistent names. We report the detection rate of RanMASK, where a lower value indicates stronger detection resistance.

TABLE VIII
DETECTION RATE OF ADVERSARIAL EXAMPLES BY RANMASK ACROSS THREE DATASETS. LOWER VALUES INDICATE HIGHER STEALTHINESS.

Attack Method	Detection Rate (RanMASK)		
	Devign	BigVul	DiverseVul
MHM	22.84%	15.21%	23.50%
ALERT	77.70%	65.67%	78.57%
CODA	43.26%	30.76%	36.58%
MOAA	34.16%	34.86%	35.41%
DIP	30.26%	14.61%	23.16%
SLODA	17.48%	8.90%	13.16%
EaTVul	21.80%	12.65%	18.64%
AttaCode	**15.42%**	**8.52%**	**11.74%**

Table VIII shows that AttaCode achieves the lowest detection rates across all datasets, with 15.42%, 8.52%, and 11.74% on Devign, BigVul, and DiverseVul, respectively. The differences among attacks largely reflect their perturbation mechanisms. Identifier substitution attacks are especially exposed by RanMASK: ALERT exceeds 65% detection rate on all datasets because extensive renaming can create context-inconsistent identifiers that are revealed during masking and reconstruction. MOAA and CODA show moderate detection rates due to partial identifier manipulation, while DIP is less affected because it mainly inserts dead code. EaTVul also exhibits moderate detection rates. Its insertion snippets generated by the LLM avoid extensive changes to existing identifiers, but they may introduce new identifiers and local code structures that are less consistent with the surrounding program context. Attacks that better preserve identifier consistency generally achieve stronger detection resistance. SLODA lowers detection rates by introducing statement-level out-of-distribution features while maintaining identifier consistency, but AttaCode further improves stealthiness on all datasets. This advantage comes from its fine-grained importance-guided transformation

strategy, which selectively applies rule-guided structural modifications without introducing conspicuous identifier inconsistencies. These results show that AttaCode provides strong resistance to automated detection without explicitly optimizing for evasion against RanMASK.

B. Cross-Model Transferability of Jailbreak Prompts

Although AttaCode uses optimized jailbreak prompts to unlock the adversarial generation capability of safety-aligned LLMs, two practical risks remain. First, a prompt that succeeds during optimization may fail after its placeholders are instantiated with complete source code and explicit transformation rules, because concrete code and rules may re-trigger the safety mechanisms of the generator LLM. Second, prompts optimized with feedback from one generator LLM may overfit to its specific alignment behavior and consequently fail on other LLMs. We investigate these questions through full-context instantiation and cross-model transfer experiments.

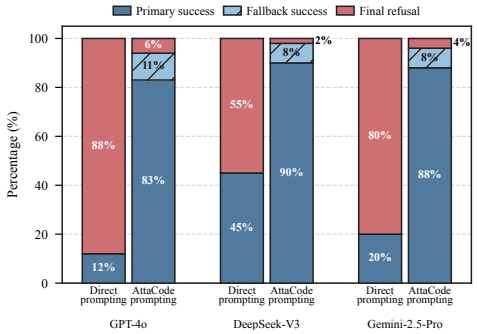


Fig. 9. Robustness of direct prompting and AttaCode prompting under concrete code and rule instantiation across three generator LLMs.

For the first experiment, we randomly sample 100 inputs to compare GPT-4o, DeepSeek-V3, and Gemini-2.5-Pro under two prompting settings. Direct prompting asks for adversarial variants without jailbreak strategies, whereas AttaCode prompting instantiates P_{opt} with the complete source code x and current rule subset $\mathcal{R}_{curr}$. Primary success means the first request generates the desired code variants. If the initial request is refused, AttaCode samples up to three fallback prompts, recording recovered requests as fallback successes and unrecovered requests as final refusals. These outcomes measure compliance with the generation request rather than detector evasion. Figure 9 shows that direct prompting achieves primary success rates of only 12%, 45%, and 20% on GPT-4o, DeepSeek-V3, and Gemini-2.5-Pro, respectively, with all remaining requests resulting in final refusals. In comparison, AttaCode prompting achieves higher primary success rates of 83%, 90%, and 88%. The fallback prompts reduce final refusals and raise overall success rates to 94%, 98%, and 96%, respectively. These results indicate that optimized prompts remain effective after instantiation with concrete program contexts and transformation requirements. The additional gains from fallback demonstrate its effectiveness in handling residual refusals across generator LLMs.

We next evaluate whether the effectiveness of optimized prompts transfers across generator LLMs without adaptation to each target. Let G_s denote the source generator LLM whose responses provide feedback for prompt optimization, and let G_t denote the target generator LLM to which the optimized prompt is transferred. The optimizer LLM A is kept fixed throughout this process. For each source–target pair, we apply the prompt optimized on G_s directly to G_t over the same 100 inputs. The prompt remains unchanged, and no re-optimization, fallback retry, or feedback from G_t is allowed. This controlled setting isolates transferability across models from gains that could otherwise come from adaptation to the target or repeated attempts. We report CR@1, defined as the proportion of requests accepted by G_t on the first attempt. As shown in Figure 10, the diagonal entries correspond to the primary success rates in Figure 9, with an average of 87.0% when each prompt is evaluated on its source generator LLM. More importantly, prompts retain substantial effectiveness across the six transfer settings, achieving an average CR@1 of 78.7%. Transfer is nevertheless asymmetric: prompts optimized on GPT-4o and Gemini-2.5-Pro achieve 96% and 92% CR@1 on DeepSeek-V3, whereas prompts optimized on DeepSeek-V3 reach 62% and 67% on GPT-4o and Gemini-2.5-Pro, respectively. These results suggest that the optimized prompts exploit alignment weaknesses shared across multiple LLMs rather than overfitting to a single generator LLM.

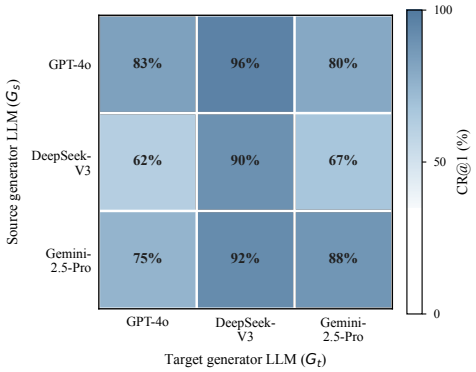


Fig. 10. Transferability of optimized jailbreak prompts across LLMs. Each cell reports CR@1 when a prompt optimized on the row source generator LLM G_s is applied to the column target generator LLM G_t.

C. Ethical Considerations and Misuse Mitigation

AttaCode is a dual-use technique because automated prompt mutation could be repurposed to weaken LLM safeguards. To mitigate this risk, we restrict all experiments to public benchmarks, predefined semantics-preserving transformations, and evaluations of locally deployed detectors. We also submitted responsible disclosure notices to affected LLM providers before public release and exclude provider-specific optimized prompts, complete refusal traces, and directly reusable bypass examples from the public artifact.

D. Threats to Validity

1) Internal validity: The primary internal threat is implementation deviations affecting reproducibility. To mitigate this risk, we use official baseline codebases when available or

reimplement the methods following the original papers, with hyperparameters set according to the authors' recommendations. Because the official EaTVul artifacts are incompatible with our function-level benchmark, we reimplemented its core attack workflow. While this adaptation may introduce minor implementation variances, the fundamental attack procedure remains identical, supporting the validity of our evaluation conclusions. We also adopt standard metrics used in prior adversarial attack studies on code models, including attack success rate (ASR) and average model queries (AMQ), to ensure consistency with existing work. Behavioral preservation is evaluated using the finite official tests of HumanEval-X. We run the complete test suite under the same compilation settings for original and transformed programs, but test success provides empirical evidence, not a formal guarantee for all inputs. Another internal threat comes from class imbalance in vulnerability datasets, which we mitigate with balanced data splitting strategies. A further reproducibility concern is reliance on multiple LLMs accessed through a commercial API aggregation service. Exact replication may be difficult if model providers update or deprecate specific versions. To mitigate this threat, we report the exact model version identifiers used in our experiments and use a fixed sampling temperature of 0.3 across all LLMs to reduce output variance. We also provide the complete system prompt and representative adversarial output logs in our code repository to support verification without repeating API calls. Finally, we evaluate AttaCode with three diverse LLMs to show that the key findings hold across providers and do not depend on any single model.

2) External validity: Threats to external validity concern the generalizability of our findings. We mitigate this threat by evaluating AttaCode on three widely used vulnerability datasets derived from real-world open-source projects and by testing multiple pre-trained vulnerability detection models covering encoder-only and encoder-decoder architectures. We further assess robustness across three state-of-the-art LLMs with strong code-related capabilities. Our scheme uses model sensitivity signals (e.g., attribution scores) to guide rule prioritization, so that perturbations are preferentially applied to influential code regions. In deployments where such signals are unavailable, importance estimation typically relies on masking-based probing and thus incurs substantially higher query cost. As demonstrated in Section V-D, leveraging gradient signals allows AttaCode to avoid this expensive exploration and reduce query overhead.

VII. Related Work

In this section, we review prior studies on adversarial attacks against code models, jailbreak prompting for aligned LLMs, and importance analysis in adversarial code generation, highlighting their limitations and differences from AttaCode.

A. Adversarial Attacks on Code Models

Adversarial attacks on code models have been studied in vulnerability detection, code completion, and summarization. Early heuristic attacks preserve functionality through identifier substitution [8], [46], structural modification [10], [24], [47], and dead code insertion [11], [12]. Although these methods achieve reasonable success rates, they often produce unnatural code and struggle to explore deeper semantic variations. With the advent of LLMs, EaTVul [14] leverages code generation capabilities to synthesize adversarial snippets for vulnerability detection evasion. However, safety alignment in commercial LLMs often causes refusals to direct adversarial code requests, so EaTVul relies on predefined templates and fragmented code contexts. As discussed in Section II-C, such segmentation can disrupt vulnerability semantics and contextual dependencies, limiting attack effectiveness. In a distinct setting, Flashboom [48] inserts high-attention decoy snippets to divert code auditors based on generative LLMs away from genuine vulnerabilities. Unlike EaTVul and AttaCode, Flashboom targets auditor outputs rather than learned vulnerability detection classifiers. AttaCode addresses these limitations by combining jailbreak prompting with full program context for more effective and natural adversarial code generation.

B. Jailbreak Prompts against Aligned LLMs

As modern LLMs increasingly incorporate safety alignment, jailbreak prompting has become a key technique for circumventing refusals [49]–[53]. Early methods mainly use fixed prompt templates or handcrafted prefixes [54], often relying on role playing, fictional framing, or benign contextualization to mask malicious intent. Later work explored more systematic strategies, including gradient optimization such as GCG [55], genetic algorithms such as AutoDAN [56], and persuasive adversarial prompts such as PAP [57]. Recent studies further emphasize iterative optimization and multi-turn interactions, with PAIR [58] using reinforcement feedback and TAP [59] exploring prompts through tree-of-thought search. From an interpretability perspective, ASJA [60] fabricates dialogue history to shift attention away from harmful keywords and increase jailbreak success. Li et al. [61] introduced MalwareBench to evaluate LLM robustness against jailbreak prompts for malicious code generation, showing that jailbreak strategies can substantially reduce refusal behavior. MalwareBench focuses on the safety robustness of LLMs used for code generation, whereas AttaCode studies jailbreak-enabled adversarial code generation against vulnerability detection models. Beyond textual jailbreaks, prompt mechanisms have also supported privacy-aware graph learning [62] and transferable multimodal dataset fingerprinting [63]. Unlike existing jailbreak studies that mainly examine harmful output elicitation or the safety robustness of code LLMs, AttaCode targets adversarial code generation under full program context. To bridge this gap, we design an automated jailbreak prompt construction system for code attacks, where a reasoning-informed assistant iteratively refines prompts based on refusal feedback and code-specific mutation paradigms.

C. Importance Analysis in Adversarial Code Generation

Identifying influential code elements is critical for guiding effective adversarial perturbations. Existing methods often estimate importance through token- or statement-level masking by replacing selected elements with placeholders, such as

[MASK], and measuring the resulting confidence drop [16], [17], [33], [47]. However, token masking requires repeated forward passes and incurs substantial computational overhead, while statement masking may lose fine-grained sensitivity. For Transformer detectors, attention weights have also been used to analyze token importance [25], although prior studies [64] suggest that attention does not always faithfully reflect prediction influence. In contrast, gradient attribution is widely used in adversarial machine learning to identify features most influential to model predictions [65]. Building on these insights, AttaCode performs attribution-guided token importance analysis and aggregates importance scores to AST nodes, enabling more precise and efficient prioritization of transformation rules.

VIII. Conclusion

In this paper, we presented AttaCode, an attribution-guided adversarial code generation scheme that combines jailbreak-enabled LLM generation with complete program context to evaluate the robustness of vulnerability detectors. The main findings are as follows: 1) aggregating token attributions at the AST level provides fine-grained guidance for prioritizing 25 semantics-preserving transformations; 2) iterative mutation optimizes placeholder-based jailbreak prompts to reduce safety refusals during subsequent generation, while the optimized prompts remain effective across generator LLMs; 3) coupling these prompts with prioritized transformations enables context-aware adversarial generation over the complete program and preserves tested behavior for 89.9% of programs after three sequential edits; and 4) AttaCode outperforms SOTA baselines, achieving an average ASR above 80% with 75%–96% fewer detector queries and the highest average ASR at every evaluated detector query budget. The generated adversarial examples also receive favorable naturalness ratings and yield the lowest automated detection rates. Adversarial fine-tuning further improves detector accuracy by 9.56–12.52 percentage points. In future work, we plan to explore multi-agent systems that dynamically integrate automated jailbreak prompt optimization with adversarial attack execution for the robustness evaluation of code intelligence systems.

References

[1] K. Sethi, S. Kumar, and H. Joshi, "Security vulnerabilities in mobile applications: Challenges, impact, and future prospects," *Journal of Android iOS Development and Testing*, vol. 10, no. 3, 2026.

[2] S. Dou, H. Zheng, J. Shan, Y. Wu, D. Zou, X. Huang, and Y. Liu, "A scalable vulnerability detection system with multi-view graph representations," *ACM Transactions on Software Engineering and Methodology*, 2025.

[3] D. Guo, S. Lu, N. Duan, Y. Wang, M. Zhou, and J. Yin, "Unixcoder: Unified cross-modal pre-training for code representation," *arXiv preprint arXiv:2203.03850*, 2022.

[4] X. Ren, Y. Ye, X. Wu, Y. Wu, and Y. Xue, "Demystifying the evolution of neural networks with bom analysis: Insights from a large-scale study of 55,997 github repositories," *arXiv preprint arXiv:2509.20010*, 2025.

[5] J. Jia, S. Ma, Y. Liu, L. Wang, and R. H. Deng, "A causality-aligned structure rationalization scheme against adversarial biased perturbations for graph neural networks," *IEEE Transactions on Information Forensics and Security*, vol. 19, pp. 59–73, 2023.

[6] G. Yang, Y. Zhou, W. Yang, T. Yue, X. Chen, and T. Chen, "How important are good method names in neural code generation? a model robustness perspective," *ACM Transactions on Software Engineering and Methodology*, vol. 33, no. 3, pp. 1–35, 2024.

[7] Z. Yang, J. Shi, J. He, and D. Lo, "Natural attack for pre-trained models of code," in *Proceedings of the 44th International Conference on Software Engineering*, 2022, pp. 1482–1493.

[8] X. Du, M. Wen, Z. Wei, S. Wang, and H. Jin, "An extensive study on adversarial attack against pre-trained models of code," in *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2023, pp. 489–501.

[9] W. Zhang, S. Guo, H. Zhang, Y. Sui, Y. Xue, and Y. Xu, "Challenging machine learning-based clone detectors via semantic-preserving code transformations," *IEEE Transactions on Software Engineering*, vol. 49, no. 5, pp. 3052–3070, 2023.

[10] A. Jha and C. K. Reddy, "Codeattack: Code-based adversarial attacks for pre-trained programming language models," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 12, 2023, pp. 14 892–14 900.

[11] C. Na, Y. Choi, and J.-H. Lee, "Dip: Dead code insertion based black-box attack for programming language model," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2023, pp. 7777–7791.

[12] F. Gao, Y. Wang, and K. Wang, "Discrete adversarial attack to models of code," *Proceedings of the ACM on Programming Languages*, vol. 7, no. PLDI, pp. 172–195, 2023.

[13] J. Jiang, F. Wang, J. Shen, S. Kim, and S. Kim, "A survey on large language models for code generation," *ACM Transactions on Software Engineering and Methodology*, vol. 35, no. 2, pp. 1–72, 2026.

[14] S. Liu, D. Cao, J. Kim, T. Abraham, P. Montague, S. Camtepe, J. Zhang, and Y. Xiang, "{EaTVul}:{ChatGPT-based} evasion attack against software vulnerability detection," in *33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 7357–7374.

[15] Y. Wan, W. Zhao, H. Zhang, Y. Sui, G. Xu, and H. Jin, "What do they capture? a structural analysis of pre-trained language models for source code," in *Proceedings of the 44th international conference on software engineering*, 2022, pp. 2377–2388.

[16] S. Zhou, M. Huang, Y. Sun, and K. Li, "Evolutionary multi-objective optimization for contextual adversarial example generation," *Proceedings of the ACM on Software Engineering*, vol. 1, no. FSE, pp. 2285–2308, 2024.

[17] X. Du, M. Wen, H. Wang, Z. Wei, and H. Jin, "Statement-level adversarial attack on vulnerability detection models via out-of-distribution features," *Proceedings of the ACM on Software Engineering*, vol. 2, no. FSE, pp. 3009–3032, 2025.

[18] Z. Jing, Z. Ying, L. Wang, S. Liang, X. Li, M. Zhang, A. Liu, X. Liu, and D. Tao, "Cogmorph: Cognitive morphing attacks for text-to-image models," *IEEE Transactions on Dependable and Secure Computing*, 2026.

[19] W. Hackett, L. Birch, S. Trawicki, N. Suri, and P. Garraghan, "Bypassing llm guardrails: An empirical analysis of evasion attacks against prompt injection and jailbreak detection systems," in *Proceedings of the The First Workshop on LLM Security (LLMSEC)*, 2025, pp. 101–114.

[20] Q. Zhang, C. Fang, B. Yu, W. Sun, T. Zhang, and Z. Chen, "Pre-trained model-based automated software vulnerability repair: How far are we?" *IEEE Transactions on Dependable and Secure Computing*, vol. 21, no. 4, pp. 2507–2525, 2023.

[21] X. She, Y. Liu, Y. Zhao, Y. He, L. Li, C. Tantithamthavorn, Z. Qin, and H. Wang, "Pitfalls in language models for code intelligence: A taxonomy and survey," *ACM Transactions on Software Engineering and Methodology*, vol. 35, no. 3, pp. 1–42, 2026.

[22] C. Wu, J. Chen, J. Li, J. Xu, J. Jia, Y. Hu, Y. Feng, Y. Liu, and Y. Xiang, "Profit or deceit? mitigating pump and dump in defi via graph and contrastive learning," *IEEE Transactions on Information Forensics and Security*, 2025.

[23] W. Li, S. Dou, Y. Wu, C. Li, and Y. Liu, "Cocl: an intelligent framework for enhancing deep learning-based vulnerability detection," *IEEE Trans. Ind. Informatics*, vol. 20, no. 3, pp. 4953–4961, 2023.

[24] Z. Tian, J. Chen, and Z. Jin, "Code difference guided adversarial example generation for deep code models," in *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2023, pp. 850–862.

[25] Z. Li, C. Zhang, M. Pan, T. Zhang, and X. Li, "Aacegen: Attention guided adversarial code example generation for deep code models," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 2024, pp. 1245–1257.

[26] S. Dou, Y. Wu, H. Jia, Y. Zhou, Y. Liu, and Y. Liu, "Cc2vec: Combining typed tokens with contrastive learning for effective code clone detection," *Proceedings of the ACM on Software Engineering*, vol. 1, no. FSE, pp. 1564–1584, 2024.

[27] S. Rahman, L. Jiang, J. Shiffer, G. Liu, S. Issaka, M. R. Parvez, H. Palangi, K.-W. Chang, Y. Choi, and S. Gabriel, "X-teaming: Multi-turn jailbreaks and defenses with adaptive multi-agents," *arXiv preprint arXiv:2504.13203*, 2025.

[28] A. Pei, Z. Yang, S. Zhu, R. Cheng, and J. Jia, "Selfprompt: Autonomously evaluating llm robustness via domain-constrained knowledge guidelines and refined adversarial prompts," in *Proceedings of the 31st International Conference on Computational Linguistics*, 2025, pp. 6840–6854.

[29] Y. Zhou, S. Liu, J. K. Siow, X. Du, and Y. Liu, "Devign: Effective Vulnerability Identification by Learning Comprehensive Program Semantics via Graph Neural Networks," in *Advances in Neural Information Processing Systems 32 (NeurIPS)*, 2019, pp. 10 197–10 207.

[30] J. Fan, Y. Li, S. Wang, and T. N. Nguyen, "A c/c++ code vulnerability dataset with code changes and cve summaries," in *Proceedings of the 17th International Conference on Mining Software Repositories*, ser. MSR '20. ACM, 2020, pp. 508–512.

[31] Y. Chen, Z. Ding, L. Alowain, X. Chen, and D. A. Wagner, "Diversevul: A new vulnerable source code dataset for deep learning based vulnerability detection," in *Proc. 26th Int. Symp. Res. Attacks, Intrusions Defenses (RAID)*, ser. RAID '23. ACM, 2023, pp. 654–668.

[32] X. Du, M. Wen, J. Zhu, Z. Xie, B. Ji, H. Liu, X. Shi, and H. Jin, "Generalization-enhanced code vulnerability detection via multi-task instruction fine-tuning," in *Findings of the Association for Computational Linguistics (ACL 2024)*. Association for Computational Linguistics, 2024, pp. 10 507–10 521, presented at ACL 2024, August 11–16.

[33] H. Zhang, Z. Li, G. Li, L. Ma, Y. Liu, and Z. Jin, "Generating adversarial examples for holding robustness of source code processing models," in *Proceedings of the Thirty-Fourth AAAI Conference on Artificial Intelligence*, ser. AAAI '20. AAAI Press, 2020, pp. 1169–1176, february 7–12, 2020.

[34] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang *et al.*, "Codebert: A pre-trained model for programming and natural languages," *arXiv preprint arXiv:2002.08155*, 2020.

[35] Y. Wang, W. Wang, S. R. Joty, and S. C. H. Hoi, "Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation," in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, ser. EMNLP 2021. Association for Computational Linguistics, 2021, pp. 8696–8708, november 7–11, 2021.

[36] OpenAI, "Hello gpt-4o," https://openai.com/index/hello-gpt-4o/, 2024, accessed: October 20, 2025.

[37] A. Liu, B. Feng, B. Xue, B. Wang, B. Wu, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan *et al.*, "Deepseek-v3 technical report," *arXiv preprint arXiv:2412.19437*, 2024.

[38] G. Comanici, E. Bieber, M. Schaekermann, I. Pasupat, N. Sachdeva, I. Dhillon, M. Blistein, O. Ram, D. Zhang, E. Rosen *et al.*, "Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities," *arXiv preprint arXiv:2507.06261*, 2025.

[39] S. Anthropic, "Model card addendum: Claude 3.5 haiku and upgraded claude 3.5 sonnet," *URL https://api. semanticscholar. org/CorpusID*, vol. 273639283, 2024.

[40] Gemini-3, "A new era of intelligence with gemini 3," https://blog.google/products-and-platforms/products/gemini/gemini-3/#gemini-3, 2025, accessed: December 30, 2025.

[41] Q. Zheng, X. Xia, X. Zou, Y. Dong, S. Wang, Y. Xue, L. Shen, Z. Wang, A. Wang, Y. Li *et al.*, "Codegeex: A pre-trained model for code generation with multilingual benchmarking on humaneval-x," in *Proceedings of the 29th ACM SIGKDD conference on knowledge discovery and data mining*, 2023, pp. 5673–5684.

[42] D. Guo, D. Yang, H. Zhang, J. Song, R. Zhang, R. Xu, Q. Zhu, S. Ma, P. Wang, X. Bi *et al.*, "Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning," *arXiv preprint arXiv:2501.12948*, 2025.

[43] A. Joshi, S. Kale, S. Chandel, and D. K. Pal, "Likert scale: Explored and explained," *Curr. J. Appl. Sci. Technol.*, vol. 7, no. 4, pp. 396–403, 2015.

[44] J. Hughes, "krippendorffsalpha: An r package for measuring agreement using krippendorff's alpha coefficient," *arXiv preprint arXiv:2103.12170*, 2021.

[45] J. Zeng, J. Xu, X. Zheng, and X. Huang, "Certified robustness to text adversarial attacks by randomized [mask]," *Computational Linguistics*, vol. 49, no. 2, pp. 395–427, 2023.

[46] N. Yefet, U. Alon, and E. Yahav, "Adversarial examples for models of code," *Proceedings of the ACM on Programming Languages*, vol. 4, no. OOPSLA, pp. 1–30, 2020.

[47] H. Zhang, Z. Fu, G. Li, L. Ma, Z. Zhao, H. Yang, Y. Sun, Y. Liu, and Z. Jin, "Towards robustness of deep program processing models—detection, estimation, and enhancement," *ACM Transactions on Software Engineering and Methodology (TOSEM)*, vol. 31, no. 3, pp. 1–40, 2022.

[48] X. Li, Y. Li, H. Wu, Y. Zhang, K. Xu, X. Cheng, S. Zhong, and F. Xu, "Make a feint to the east while attacking in the west: Blinding llm-based code auditors with flashboom attacks," in *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2025, pp. 576–594.

[49] H. Xu, X. Zhang, Z. Chen, M. Yao, and Y. Liu, "Prompting for trouble: An empirical analysis of jailbreaking large language models," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*. ACL, 2023, pp. 1–10.

[50] X. Li, R. Wang, M. Cheng, T. Zhou, and C.-J. Hsieh, "Drattack: Prompt decomposition and reconstruction makes powerful llm jailbreakers," *arXiv preprint arXiv:2402.16914*, 2024.

[51] J. Jia, J. Yu, D. Wu, C. Wu, H. Zhu, and L. Wang, "Prompt as a double-edged sword: A dynamic equilibrium gradient-assigned attack against graph prompt learning," in *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*, 2025, pp. 1049–1060.

[52] G. Chen, F. Song, Z. Zhao, X. Jia, Y. Liu, Y. Qiao, W. Zhang, W. Tu, Y. Yang, and B. Du, "Audiojailbreak: Jailbreak attacks against end-to-end large audio-language models," *IEEE Transactions on Dependable and Secure Computing*, 2026.

[53] Z. Chen, C. Li, C. Li, X. Zhang, L. Zhang, and Y. He, "Jailbreaking llms & vlms: Mechanisms, evaluation, and unified defense," *arXiv preprint arXiv:2601.03594*, 2026.

[54] A. Wei, N. Haghtalab, and J. Steinhardt, "Jailbroken: How does llm safety training fail?" *Advances in Neural Information Processing Systems*, vol. 36, pp. 80 079–80 110, 2023.

[55] A. Zou, Z. Wang, N. Carlini, M. Nasr, J. Z. Kolter, and M. Fredrikson, "Universal and transferable adversarial attacks on aligned language models," *arXiv preprint arXiv:2307.15043*, 2023.

[56] X. Liu, N. Xu, M. Chen, and C. Xiao, "Autodan: Generating stealthy jailbreak prompts on aligned large language models," *arXiv preprint arXiv:2310.04451*, 2023.

[57] Y. Zeng, H. Lin, J. Zhang, D. Yang, R. Jia, and W. Shi, "How johnny can persuade llms to jailbreak them: Rethinking persuasion to challenge ai safety by humanizing llms," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024, pp. 14 322–14 350.

[58] P. Chao, A. Robey, E. Dobriban, H. Hassani, G. J. Pappas, and E. Wong, "Jailbreaking black box large language models in twenty queries," in *2025 IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*. IEEE, 2025, pp. 23–42.

[59] A. Mehrotra, M. Zampetakis, P. Kassianik, B. Nelson, H. Anderson, Y. Singer, and A. Karbasi, "Tree of attacks: Jailbreaking black-box llms automatically," *Advances in Neural Information Processing Systems*, vol. 37, pp. 61 065–61 105, 2024.

[60] X. Du, F. Mo, M. Wen, T. Gu, H. Zheng, H. Jin, and J. Shi, "Multi-turn jailbreaking large language models via attention shifting," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, no. 22, 2025, pp. 23 814–23 822.

[61] H. Li, H. Gao, Z. Zhao, Z. Lin, J. Gao, and X. Li, "Llms caught in the crossfire: Malware requests and jailbreak challenges," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2025, pp. 27 833–27 848.

[62] J. Jia, J. Song, J. Yu, J. Guo, X. Jia, D. Wu, Y. Yuan, and G. Cheng, "Pagpl: Privacy-aware graph prompt learning scheme via adaptive perturbation-estimated topology recovery," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 40, no. 27, 2026, pp. 22 247–22 255.

[63] W. Zhang, J. Jia, X. Jia, Y. Huang, X. Li, C. Wu, and L. Wang, "Patfinger: Prompt-adapted transferable fingerprinting against unauthorized multimodal dataset usage," in *Proc. 48th Int. ACM SIGIR Conf. Res. Develop. Inf. Retrieval (SIGIR)*, 2025, pp. 403–413.

[64] M. Saad and T. Sharma, "Naturalness of attention: Revisiting attention in code language models," in *Proc. ACM/IEEE Int. Conf. Softw. Eng., New Ideas Emerg. Results (ICSE-NIER)*, 2024, pp. 107–111.

[65] X. Li, G. Meng, S. Liu, L. Xiang, K. Sun, K. Chen, X. Luo, and Y. Liu, "Attribution-guided adversarial code prompt generation for code completion models," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 2024, pp. 1460–1471.